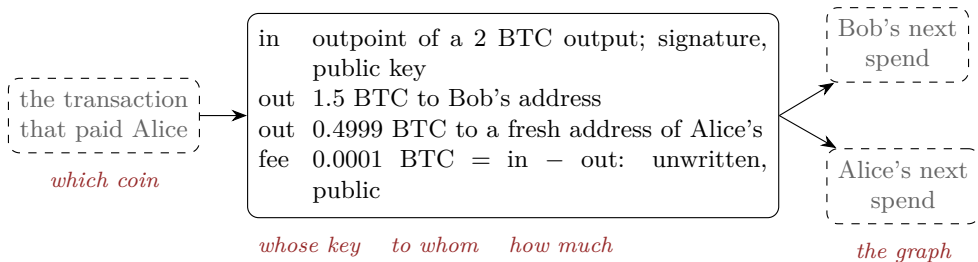


Part A — What Bitcoin publishes, and what privacy must mean

Alice pays Bob in Bitcoin: what the chain publishes

all four

Write



Say

Nobody wrote “Alice” or “Bob” on the chain, but which coin, whose key, to whom, how much and the spend graph are all in the record; the unwritten fee is public. Address reuse merges a history; the non-round change output to a fresh address is usually the payer’s. The shielded running example has this shape, with none of the labels.

The validator's four checks

all four

Write

```
validate(tx):
```

```
    for input in tx.inputs:
```

```
        coin = utxo[input.prev_out]
```

```
        check sig(input.sig, coin.script)
```

```
        delete utxo[input.prev_out]
```

```
    check sum(in) >= sum(out)
```

(1) coin exists

(2) authorised

(3) not spent twice

(4) conserved

reads the outpoint

reads script and key

reads the outpoint

reads every amount

```
fee =  $\Sigma$  in -  $\Sigma$  out
```

Say

In Bitcoin the whole state is one table, the set of unspent outputs; a spend is a lookup, a signature check and a deletion. Every check reads a plaintext field of the transaction or of the output it names: nothing is inferred. The fee is check 4's slack: not a field, yet public to the satoshi. These four numbers are the spine of the workshop; every later board names the check it serves.

Hide a field, lose a check

all four

Write

- hide the amount → check 4 dies: nothing to add up
- hide the input reference → checks 1 and 3 die: nothing to look up, nothing to delete
- hide the key → check 2 dies: nothing to verify against
- hide the receiver → nothing dies here: the output's script is what check 2 reads when Bob spends it, so that spend's check 2 dies

Privacy, for this workshop: hide the sender, the receiver and the amount, while a stranger still performs all four checks.

Say

Every mechanism to come answers the same pair of questions: which of the four checks does it restore, and why can the node no longer do it the Bitcoin way?

What a privacy design must supply

all four

Write

<i>the validator must</i>	<i>the tool</i>	<i>Bitcoin's version</i>
accept a claim about data it cannot see	a zero-knowledge proof: the statement public, the witness (the prover's secret input, not Bitcoin's witness field) private	look at the data
know the hidden data cannot change under it	a commitment: hiding, and binding	the output, in the clear
notice the same hidden coin a second time	a nullifier: a keyed, one-way tag of the coin, published at spend	delete it from the set
add up amounts it cannot read	a homomorphic commitment: sealed envelopes that add	add the numbers
<i>the proof checks the claim; the other three tools make the claim about something fixed, unrepeatable, summable.</i>		

Write

$$([v_1] V + [r_1] R) + ([v_2] V + [r_2] R) = [v_1 + v_2] V + [r_1 + r_2] R$$

holds modulo the group order; the 64-bit range check lives inside the proof.

Say

Two commitments add without either being opened: the two sealed envelopes combine into an envelope for the sum of the values, with the sum of the randomness. It is the Confidential Transactions trick: amounts hidden, their sum still checkable.

Notes versus UTXOs: what a coin is

authorised, conserved

Write

	<i>Bitcoin UTXO</i>	<i>Zcash note</i>
owner	a script, usually one key hash, on chain	an address $(d, \mathbf{pk}_d)$, inside the note
value	the amount, on chain	$v \in [0, 2^{64})$ zatoshi, inside the note
spending condition	script: signatures, timelocks, hashlocks	a key, full stop — no script, no timelocks, no HTLCs

Say

In Zcash both owner and value are fields of the note, and the note is never published. Threshold authority exists and is invisible on chain: it is FROST, later in the workshop; on chain there is one ordinary signature under one per-spend key.

Notes versus UTXOs: what the chain records

exists, not twice

Write

	<i>Bitcoin UTXO</i>	<i>Zcash note</i>
on-chain identifier	the outpoint	a commitment cm to the note, stored as cmx
spent marker	removed from the UTXO set	a nullifier nf , published at spend
set structure	one set: insert, delete	an append-only tree of cmx , depth 32, and a set of nf : append only, both

the note never touches the chain.

Say

Nothing is ever deleted. Recorded are a commitment to the note, an encrypted copy for its recipient, and at spend a tag derived from it.

Write

A shielded spend is three claims about a note nobody sees:

1. *it exists* — a commitment to it is a leaf under a public root, proved with leaf, position and path all private;
2. *it is spent now* — its serial number, the nullifier, is revealed, and the chain refuses to see it twice;
3. *the books balance* — commitments to the amounts add up to the declared net flow.

$$R_{\text{mem}} = \left\{ \left(\underbrace{rt}_{\text{public}} ; \underbrace{(\text{cmx}, i, \text{path})}_{\text{private}} \right) : \text{Verify}(rt, i, \text{cmx}, \text{path}) = \text{accept} \right\}$$

Say

A Bitcoin spend names its coin and signs; a shielded spend names nothing: it proves line 1, reveals line 2 and signs for line 3, under a key the proof ties to the note (check 2). Everything else in the workshop is machinery for these three lines and that signature.

Four checks, four mechanisms

all four

Write

	<i>the check</i>	<i>Bitcoin</i>	<i>shielded Zcash</i>
(1)	coin exists	look the outpoint up in the UTXO set	a commitment in an append-only tree; a membership proof under a public root
(3)	not spent twice	delete it from the set	a nullifier, published once, refused the second time
(4)	conserved	add the amounts up	value commitments that add; a binding signature over their sum
(2)	authorised	run the script over the signature	key knowledge proved inside the proof; a signature under a re-randomised key

Say

Each part of the workshop reopens with both questions: which check, and why not the Bitcoin way (settled on “Hide a field, lose a check”). Checks 1 and 3 come first, with the data model; checks 4 and 2 follow.

Write

1. **The toolbox** — each primitive from the Bitcoin instance the room already holds.
2. **The data model** — notes, commitments, the tree, the nullifier: checks 1 and 3.
3. **Keys and addresses** — who can spend, who can see.
4. **The Action statement** — what the proof proves, clause by clause; then a break.
5. **Balance and authority** — checks 4 and 2, on the running example.
6. **The proof system** — from a statement to a few kilobytes.
7. **Delivery and light clients** — how Bob finds his money.
8. **The transaction and the turnstile** — the bytes on the chain, and the pools.
9. **Consensus** — why one fork wins, what the chain must remember.
10. **Limits, frontier, takeaways.**

one example threads it: Alice pays Bob 1.5 ZEC from a 2 ZEC note.

Ask

Which of the four checks does a nullifier serve, and what breaks if two different notes could share one?

Answer

- Check 3, not spent twice. The nullifier is “delete from the UTXO set” without naming the coin: published once, refused the second time.
- Two notes, one nullifier: whichever is spent first burns the other, and the second owner’s honest spend is refused as a double spend — an attack on earlier designs, in which forged notes could be made to share one.
- How the chain makes sure no two accepted notes can share one — every new note inherits, as its seed, the serial of the note spent beside it — is Part C’s business.

Part B — The toolbox, from Bitcoin's primitives up

Hash functions: one primitive, three notions

all four

Write

1. $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$ — fixed, public, compresses
2. preimage: given y , find x' with $H(x') = y$
3. second preimage: given x , find $x' \neq x$ with $H(x') = H(x)$
4. collision: find any $x \neq x'$ with $H(x) = H(x')$ — strongest; commitments, Merkle trees, Fiat–Shamir
5. generic: collision $\approx 2^{n/2}$ (birthday), preimage $\approx 2^n$; $n = 256 \Rightarrow 2^{128}$; RIPEMD-160 $\Rightarrow 2^{80}$
6. Bitcoin: SHA-256d (headers, txids); $\text{hash160} = \text{RIPEMD160} \circ \text{SHA256}$ (addresses)
7. Zcash: keeps all of these (transparent) + three shielded hashes

Say

In the collision game the adversary picks both inputs, so it is the strongest demand — the one commitments, Merkle trees and Fiat–Shamir need. The generic costs set the digest sizes. Nothing is taken away: Zcash keeps every Bitcoin hash in its transparent layer and adds three for the shielded one.

Ask

Which of the three notions do commitments and Merkle trees lean on?

Answer

Collision resistance: the adversary picks both inputs.

Zcash's hashes: SHA-256 is not replaced

all four

Write

<i>hash</i>	<i>kind</i>	<i>deployed use</i>
SHA-256d	bit-oriented, Bitcoin's	header difficulty filter (after Equihash); transparent layer: legacy txids, Base58Check
BLAKE2b	bit-oriented; personalised natively, the key prepended to the input	PRFexpand (the key tree); note-encryption KDF; ZIP-244 digests (txid, sighash)
Poseidon	algebraic: $x \mapsto x^5$ over $\mathbb{F}_{p_{\text{Pallas}}}$	the nullifier PRF, inside the circuit
Sinsemilla	algebraic: curve additions on Pallas	note commitment; Merkle node hash, both inside the circuit

Write

1. Bitcoin's hashes stay where Bitcoin put them
2. shielded: +1 bit-oriented (off the circuit), +2 algebraic (inside it)

Say

SHA-256d is not replaced: it keeps the header difficulty filter and the transparent layer. The split is by where the work happens: BLAKE2b off the circuit; Poseidon and Sinsemilla inside it.

Why in-circuit hashing wants algebraic hashes

exists, not twice

Write

1. proof = polynomial constraints over a prime field; field multiplication = 1 constraint; Boolean operation = several + range checks
2. SHA-256, BLAKE2: 32-/64-bit words, XOR, shuffles $\Rightarrow$ tens of thousands of constraints per evaluation
3. Poseidon round: + constants; x^5 (all elements in a full round, one in a partial); $\times$ fixed matrix. 2-to-1 hash $\approx$ a few hundred constraints. Orchard: width 3, 8 full + 56 partial
4. Sinsemilla, per 10-bit chunk: 1 lookup (table of $2^{10} = 1024$ points) + 2 incomplete additions; collision $\Rightarrow$ discrete log

Say

A zero-knowledge proof re-expresses every step of a computation as polynomial constraints over a prime field. SHA-256 and BLAKE2 are foreign to the field; Poseidon keeps the only non-linear step at x^5 ; Sinsemilla is curve arithmetic, native to the circuit. Bitcoin never hashes inside a proof; the first shielded generation did, with SHA-256, and its prover paid for it.

Write

1. $y^2 = x^3 + b$ over a prime field; prime-order group, cofactor 1; discrete log = the standing assumption
2. secp256k1: $b = 7$, $p = 2^{256} - 2^{32} - 977$, 256-bit prime order
3. Pallas, Vesta: $b = 5$, two 255-bit primes p_{Pallas} , p_{Vesta} ; $\#Pallas = p_{\text{Vesta}}$, $\#Vesta = p_{\text{Pallas}}$; ≈ 126 bits of DL security
4. generators: hashed to the curve (secp256k1's base point: a SEC 2 constant)
5. Pallas: every point in an Action; Vesta: the proof's polynomial commitments

Say

The scalar field of each Pasta curve is the base field of the other. Every Pallas and Vesta generator the protocol uses is hashed to the curve, so no relation between any two is known to anyone and there is no ceremony for this stack; the Sapling pool still rests on its Groth16 MPC.

Why not secp256k1: two-adicity

all four

Write

<i>field</i>	<i>bits</i>	<i>two-adicity of $p - 1$</i>	<i>largest power-of-two domain</i>
Pallas base field	255	32	2^{32}
Vesta base field	255	32	2^{32}
secp256k1 base field	256	1	2
secp256k1 scalar field	256	6	64

1. 2^k -th roots of unity exist iff $2^k \mid p - 1$; largest such $k =$ two-adicity
2. Orchard circuit: 2^{11} rows
3. secp256k1 rows: our own computation, not a volume figure

Say

A proof system of the PLONK family interpolates each circuit column over the 2^k -th roots of unity of its field. Neither secp256k1 field offers a domain of 2^{11} ; the Pasta primes were searched for with two-adicity 32 and the cycle as design goals.

Ask

Which secp256k1 field could host 2^{11} rows?

Answer

Neither: the largest domains are 2 and 64.

Write

1. $c = \text{Commit}(m; r)$, trapdoor r ; hiding: two candidate m indistinguishable from c ; binding: no c opens to two m
2. not both perfect; deployed: perfectly hiding, binding under DL
3. $\text{Commit}(v; r) := [v] V + [r] R$, with V, R hashed to Pallas
4. uniform $r \Rightarrow$ uniform point; two openings $\Rightarrow \log_V R$
5. Bitcoin: CT's envelope; an address = hash commitment to a key/script, opened at spend
6. Zcash: cv seals the value change (Pedersen, adds); cm seals the whole note (algebraic hash $+ [rcm] H_D$)

Say

Hiding: for uniform r the point is uniform whatever v is. Binding: two openings of one c solve $\log_V R$, which nobody holds. The note commitment's blinding term $[rcm]H_D$ has the same Pedersen shape over its own hashed generator, so the circuit recomputes it cheaply.

Ask

Can the envelope be perfectly hiding and perfectly binding at once?

Answer

No; the deployed ones are perfectly hiding and binding under the discrete logarithm.

Write

1. $\text{Commit}(v_1; r_1) + \text{Commit}(v_2; r_2) = \text{Commit}(v_1 + v_2; r_1 + r_2)$

2.

$$\sum \text{Commit}_{\text{in}} - \sum \text{Commit}_{\text{out}} = \left[\sum v_{\text{in}} - \sum v_{\text{out}} \right] V + [\bar{r}] R$$

3. V term vanishes iff values balance — modulo the group order; range check on every value closes the wrap
4. CT (Elements): same equation on chain; range proof beside each commitment; receiver told the opening; amounts hidden, keys and graph clear
5. here: 64-bit range check + opening $(v, \text{rcv}) =$ clauses of one ZK proof; net trapdoor $\bar{r} \rightarrow$ a signing key

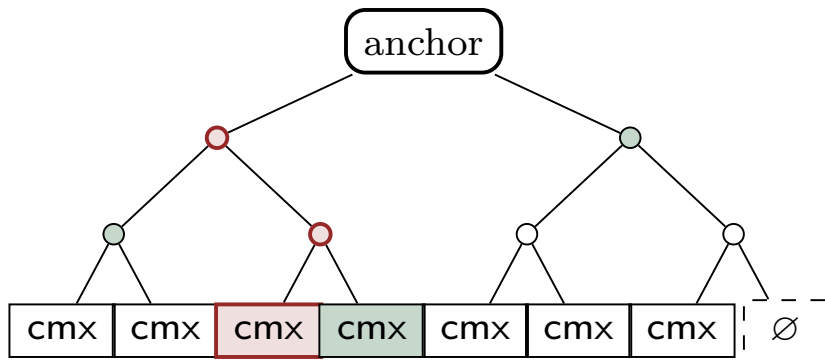
Say

Sum the inputs, subtract the outputs: the V term carries $\sum v_{\text{in}} - \sum v_{\text{out}}$ and vanishes exactly when the values balance. Confidential Transactions checks this equation on the chain; here it moves inside the proof, and the net trapdoor $\bar{r}$ becomes a signing key (board *Preview: a signature under a key nobody holds unless the values balance*).

Merkle trees: from the txid tree to the note commitment tree

exists

Write



red: the leaf and its route, private; green: the siblings, private; the anchor alone is public

Write

1. Bitcoin: txids pairwise up to the header root; SPV proof = leaf + one sibling per level; the leaf (txid) is public
2. Zcash: leaves = cmx in creation order; depth 32, room for 2^{32} notes; node hash Sinsemilla, tagged with its layer; root after each block = that block's *anchor*
3. membership proof: 32 siblings + 32 direction bits, verified by 32 hashes — inside the proof

Say

Bitcoin's SPV proof is the leaf plus one sibling per level, and the leaf is public. Here the 32 hashes are recomputed inside the proof: the leaf, its route and the siblings stay private; the anchor alone is public.

What changes: append only, historical roots, private path

exists

Write

1. append only: one leaf at a time; frontier ≤ 32 carried labels; root updated in 32 hashes; nothing deleted — “deleted” = a separate set
2. history binding: proof against the root at insertion valid forever; consensus accepts the anchor of any earlier main-chain block
3. private path: (leaf, index, siblings) = witness; anchor = statement
- 4.

$$R_{\text{mem}} = \{ (rt; (\text{cmx}, i, \text{path})) : \text{Verify}(rt, i, \text{cmx}, \text{path}) = \text{accept} \}$$

5. Bitcoin SPV names the txid; this proof names only the root

Say

Revealing (leaf, index, siblings) would say which note is spent, so the path is a witness and only the anchor a statement. An SPV proof names the txid it proves; this one hides the leaf among every leaf beneath it.

Write

1. F_k pseudorandom: secret uniform k ; no efficient adversary with oracle access (any inputs, adaptively) tells F_k from a random function
2. algorithm public; only k hidden; adversary sees input–output pairs, never k
3. Bitcoin: HMAC-SHA512 in BIP-32 child derivation
4. Zcash, off circuit: BLAKE2b personalised = PRFexpand; $sk \rightarrow ask, nk, r_{ivk}$; later the note's randomness from its seed
5. Zcash, in circuit: $F_{nk}(\rho) = \text{PoseidonHash}(nk, \rho)$ over $\mathbb{F}_{p_{\text{Pallas}}}$; width 3, S-box x^5 , 8 full + 56 partial rounds

Say

Bitcoin already trusts a PRF to make BIP-32 child keys look unrelated. Off the circuit, one PRFexpand call per sub-secret builds the key ladder (Part D). Inside it, Poseidon is keyed by its first input: nk first, ρ second.

What a nullifier needs from a PRF

not twice

Write

1.

$$\text{nf} = \text{Extract}([F_{\text{nk}}(\rho) + \psi] G^{\text{nf}} + \text{cm})$$

2. Extract: the x -coordinate; ρ, ψ : per-note randomness fixed at creation; G^{nf} : a hashed generator
3. uniqueness: all inputs fixed once the note exists $\Rightarrow$ deterministic; two notes, one tag $\Rightarrow$ DL relation on Pallas; no PRF property used (forger holds nk)
4. unlinkability: without nk, $[F_{\text{nk}}(\rho) + \psi] G^{\text{nf}}$ masks cm — Poseidon PRF or DDH on Pallas; either suffices
5. Bitcoin: spent marker = the outpoint, unique and public

Say

Uniqueness uses no PRF property: every input is fixed once the note exists, and the forger holds nk . Unlinkability does: without nk the masking term looks random. Bitcoin's spent marker is the outpoint, unique and public; a nullifier must be unique *and* say nothing — two separate properties.

Ask

Which of the two properties uses the PRF?

Answer

Only unlinkability; uniqueness is discrete-log structure, and the forger holds nk anyway.

Key agreement: the ECDH of silent payments

delivery, not a check

Write

1. base point P ; secret a , public $[a]P$; Agree(a, Q) = $[a]Q$; $[a]([b]P) = [b]([a]P)$
2. BIP-352: sender's input key + receiver's scan key $\rightarrow$ shared secret $\rightarrow$ tweaks the output key; nothing encrypted
3. Zcash address: $\text{pk}_d = [\text{ivk}] \text{g}_d$, g_d hashed from the diversifier
4. sender: ephemeral esk (from the note's seed, PRFexpand); publishes $\text{epk} = [\text{esk}] \text{g}_d$
5. $s = [\text{esk}] \text{pk}_d = [\text{ivk}] \text{epk}$
6. cofactor 1 $\Rightarrow$ only on-curve, non-identity check on epk

Say

BIP-352 encrypts nothing; the receiver scans for outputs that decode under the shared secret. Static-ephemeral suits a chain: the recipient was never online, and the sender learns nothing beyond the address.

Encryption on top: KDF, AEAD, wrong-key rejection

delivery, not a check

Write

1.

$$K_{\text{enc}} = \text{BLAKE2b-256}(\text{"Zcash_OrchardKDF"}; s \parallel \text{epk})$$

2. AEAD: ChaCha20-Poly1305; 256-bit key; nonce all zero; associated data empty; 128-bit tag; each key encrypts exactly once
3. wrong key $\Rightarrow \perp$, except with probability $\approx 2^{-128}$; not key-committing $\Rightarrow$ recompute cm and compare
4. Bitcoin: nothing encrypted on chain; here recipient, value, memo exist on chain only inside the ciphertext
5. two ciphertexts per Action: to the recipient; to the sender's own outgoing key

Say

Folding **epk** into the KDF binds the key to this ephemeral. Wrong-key rejection is the property the wallet spends: it lets a wallet try every output on the chain with its own key.

Ask

Why is an all-zero nonce safe?

Answer

Each key encrypts exactly once.

Write

1. Bitcoin: ECDSA (r, s) , secp256k1, DER + sighash-type byte; Taproot: BIP-340 Schnorr, $s = k + e d$, $e = H(x(R) \parallel x(X) \parallel m)$
2. Zcash transparent: the same ECDSA (sighash as message, DER, low- s); no Taproot, no BIP-340
3. Zcash shielded: RedPallas = Schnorr over Pallas, base point P a parameter; $X = [x]P$
- 4.

Sign: $r = H(T \parallel X \parallel m)$ for 80 fresh random bytes T , $R = [r]P$,
 $c = H(R \parallel X \parallel m)$, $s = r + c x$;

Verify: $[s]P = R + [c]X$

H : BLAKE2b-512, personalised, reduced mod the group order

5. two instantiations, differing only in P : $G^{\text{SpendAuth}}$ and the value-commitment base R

Say

The transparent layer keeps Bitcoin's ECDSA; RedPallas is Schnorr over Pallas with the base point a parameter, and the next three boards use one instantiation each.

The need: one spend key per wallet, not per coin

authorised

Write

1. Bitcoin: one key per output; reuse = a choice; each reuse links two payments
2. Zcash: one spend validating key per wallet, $ak = [ask] G^{\text{SpendAuth}}$
- 3.

$$ivk = \text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk), \quad pk_d = [ivk] g_d$$

4. un-randomised spend $\Rightarrow$ publishes ak for every note the wallet ever spends: worse than address reuse, structural
5. fresh disguise per spend; Schnorr key $= [x]P$, homomorphic $\Rightarrow$ shiftable without the secret

Say

Every diversified address shares ak , so an un-randomised spend would publish it for every note the wallet ever spends. A Schnorr key is the homomorphic image $[x]P$ of its secret, so it can be shifted without the secret.

Write

1. fresh uniform α per spend — added, never multiplied in
- 2.

$$\text{rsk} := \text{ask} + \alpha, \quad \text{rk} := \text{ak} + [\alpha] G^{\text{SpendAuth}} = [\text{rsk}] G^{\text{SpendAuth}}$$

3. consistency: (rsk, rk) an ordinary key pair
4. unlinkability: uniform $\alpha \Rightarrow \text{rk}$ uniform, whatever ak — perfect, no assumption; α fresh and secret
5. signing needs ask and α ; circuit proves rk re-randomises the ak in the note's address; node checks the signature under rk
6. BIP-32 non-hardened child = $\text{pubkey} + [\text{tweak}] G$; tweak recomputable from the xpub — α never published

Say

For uniform α the key rk is uniform whatever ak is — perfect, no assumption — provided α is fresh and stays secret. A non-hardened BIP-32 child is $pubkey + [tweak] G$ too, but anyone holding the parent $xpub$ recomputes its tweak; α is never published.

Ask

Who can recompute rk from ak ?

Answer

Nobody without α — unlike the BIP-32 tweak, which anyone holding the xpub recomputes.

Preview: a signature under a key nobody holds unless the values balance

conserved

Write

1.

$$\mathbf{bvk} = \sum \mathbf{cv} - [\mathbf{valueBalance}] V \quad (\text{public})$$

2. balanced $\Rightarrow V$ terms cancel, $\mathbf{bvk} = [\sum \mathbf{rcv}] R$, author knows $\mathbf{bsk} = \sum \mathbf{rcv}$

3. not balanced $\Rightarrow \mathbf{bvk} = [\beta] R$ would reveal $\log_R V$ — nobody holds it

4. binding signature: RedPallas under $\mathbf{bsk}$, verified under $\mathbf{bvk}$, over the sighash; knowing the log = “the values balance”

5. range checks in the proof: balance mod group order $\rightarrow$ balance over the integers

Say

Every group element is some multiple of R ; what matters is who can compute the multiplier. Bitcoin’s check 4 reads the amounts; here they stay sealed and a signature certifies their sum.

Write

1. relation R of pairs (x, w) : statement x public, witness w private; claim: “I know w with $(x, w) \in R$ ”
2. complete: honest prover with w convinces
3. sound: no prover convinces on a false x — *argument* if only efficient provers are excluded (price of succinctness)
4. zero knowledge: a simulator without w reproduces the verifier’s whole view; nothing beyond x leaks
5. knowledge sound: whoever convinces can be rewound into surrendering w
6. “some spendable note exists” — true for everyone; “I hold one” — the claim

Say

Knowledge soundness is the property this protocol needs. Zero knowledge says nothing beyond x leaks, no more.

Write

1. Fiat–Shamir: $\text{challenge} = H(\text{statement} \parallel \text{transcript so far})$; one string carries the exchange; anyone re-derives the challenges
2. proof of knowledge + Fiat–Shamir = signature
3. Schnorr = proof of knowledge of x in $X = [x]G$, message folded into the challenge hash
4. Bitcoin: one equation per signature; here: one fixed circuit's verifying key against the statement, per proof

Say

Fiat–Shamir removes the interaction; a proof of knowledge with the message folded into the challenge hash is a signature, and the room's Schnorr signature is exactly this.

Write

1. SNARK = the same move on a whole statement: “this note is in the tree, this nullifier is its, these values balance”
2. proof size and verifier time sublinear in the computation
3. Halo 2 as deployed: the proof is succinct; the verifier’s linear part is shared across a batch
4. succinct $\Rightarrow$ argument: proof shorter than the witness cannot pin it down against an unbounded prover; soundness against efficient provers, under an assumption

Say

Succinctness forces an argument: soundness holds against efficient provers, under an assumption. The statement in Part E; the proof system in Part G.

Proof sizes, with units

all four

Write

<i>generation</i>	<i>system</i>	<i>one proof covers</i>	<i>bytes</i>	<i>running example</i>
Sprout	BCTV14	one JoinSplit: 2 in, 2 out	296 B	1 JoinSplit, 296 B
Sapling	Groth16	one Spend or one Output description	192 B each	1 spend + 2 outputs, 576 B
Orchard	Halo 2	the whole bundle, a Actions	$2720 + 2272 a$ B	$a = 2$ in the Ironwood pool, 7264 B

1. $a = 1$: 4992 B; Alice: one proof for both Actions, $4992 + 2272 = 7264$ B
2. curves: BN254 $\rightarrow$ BLS12-381 $\rightarrow$ Pallas/Vesta; pairing proofs constant-size; growth to Halo 2 = the price of no trusted setup
3. Sprout's row: its original system; it later moved to Groth16

Say

Pairing proofs are constant-size; the growth to Halo 2 is the price of no trusted setup for this stack — the Sapling pool still rests on its Groth16 MPC.

Write

1. transcript reveals nothing beyond the public inputs: anchor, cv, nf, rk, cmx, flags — that is all of ZK
2. necessary, not sufficient: the public fields must not link on their own; the proof cannot help them
3. cm, cv: hiding — uniform for a uniform trapdoor
4. nf: keyed PRF — random-looking without nk
5. rk: re-randomisation — uniform for a uniform α
6. epk, ciphertexts: key agreement + KDF + AEAD
7. Bitcoin signature hides its key's secret, nothing else; here every field is a disguise, the proof certifies they are consistent

Say

The proof hides the witness; the hiding of the published fields is the commitments', the PRF's, the re-randomisation's and the encryption's work.

The instantiation table

all four

Write

<i>primitive</i>	<i>Bitcoin</i>	<i>Zcash</i>	<i>in the Action; part</i>
hash	SHA-256d, hash160	BLAKE2b; Poseidon; Sinsemilla	keys, nf, cmx; C, D
curve	secp256k1	Pallas (Action), Vesta (proof)	every point; G
commitment	CT Pedersen; key/script hash	cv Pedersen; cm Sinsemilla	cv, cmx; C, E
Merkle tree	txid tree	commitment tree, depth 32	anchor, A3; C, E
PRF	HMAC-SHA512	PRFexpand; Poseidon	nf, A5; C, D
key agreement	BIP-352 ECDH	ECDH to g_d	epk; F
encryption	none	ChaCha20-Poly1305	ciphertexts; F
signature	ECDSA; BIP-340	ECDSA (transparent); RedPallas	rk, two signatures; E, E'
ZK proof	none	Halo 2 zk-SNARK per bundle	the proof; E, G

Say

Every row is a board already on the wall. The last column is the map for the rest of the workshop: the data model in Part C, the key ladder in Part D, the statement in Part E, balance in Part E', delivery in Part F, the proof system in Part G.

Ask

Checkpoint — why can't the nullifier simply be the hash of the note commitment?

Answer

- The commitment is public: it is the leaf. Anyone could hash every leaf and match the results against published nullifiers — each spend would point at its mint, and the transaction graph would return.
- Uniqueness would survive (distinct commitments, distinct hashes); unlinkability would not. The two properties are separate, and the second needs a secret.
- Deployed: $\text{nf} = \text{Extract}([F_{\text{nk}}(\rho) + \psi] G^{\text{nf}} + \text{cm})$ — keyed by nk , so only its holder can compute it; the discrete-log structure keeps uniqueness provable and defends unlinkability twice over.

Part C — The data model

A note: the shielded unit of value

exists

Write

$$\mathbf{n} := (d, \mathbf{pk}_d, v, \rho, \psi, \mathbf{rcm})$$

field	job	in Bitcoin
$(d, \mathbf{pk}_d)$	the recipient's diversified address: an 88-bit diversifier and a transmission key (Part D)	scriptPubKey
v	the value, $0 \leq v < 2^{64}$ satoshi	amount
ρ	a per-note uniqueness seed, fixed at creation	none
ψ	nullifier salt, derived from the note seed rseed	none
rcm	commitment trapdoor, derived from rseed ; for an Ironwood note, last and from every other field (Part F)	none

Bitcoin publishes every field of an output in clear; every field of a note stays with its owner.

Say

Every one of the four checks is rebuilt on something else; this part builds it.

A note: content and machinery

exists

Write

- content: $(d, \mathbf{pk}_d), v$
- machinery: $\rho, \psi \rightarrow$ the nullifier; $\mathbf{rcm} \rightarrow$ hiding
- Alice: $v = 2 \text{ ZEC} = 2 \cdot 10^8$ zatoshi, Ironwood pool, lead byte 0x03

Say

- The seeds ρ and ψ exist only to make the note's eventual nullifier unique and unpredictable; the trapdoor $\mathbf{rcm}$ makes the commitment hiding.
- Alice's note, the running example: two ZEC at one of her own addresses, in the Ironwood pool; its plaintext carries lead byte 0x03 (Part F).
- The note itself never touches the chain; what does comes on the next boards.

Vocabulary: one protocol, two pools

exists

Write

- Orchard *protocol* = the cryptography: curves, Sinsemilla, Action circuit, notes, commitments, nullifiers, keys, note encryption
- *pool* = one note commitment tree + one nullifier set + one public balance row
- Ironwood pool: current; all new value enters here; lead byte 0x03; tree began empty at activation
- Orchard pool: legacy; sealed spend-only by three rules
- one asset (ZEC), one chain, one transaction format
- Bitcoin: P2WPKH beside P2PKH, plus a public total per type

Say

- The protocol is the shared cryptography; a pool is state, its balance row being the chain value pool balance. Two pools run one protocol.
- Bitcoin analogue: an output type on the same chain, if Bitcoin also tracked the total value held in each type.

Write

1. **No issuance:** coinbase carries an *empty* Orchard component
2. **No circulation:** every Orchard-pool Action has `enableCrossAddress = 0` (in the verifying key, selected by branch)
3. **No entry:** per transaction, $v_{\text{balance}}^{\text{Orchard}} \geq 0$

Say

- Three consensus rules bind every transaction regardless of its version: a spend may pay only its own receiver; net value may leave the pool, never enter.
- Rule two lives in the Action-circuit verifying key: no version bypasses it.
- Why: legacy notes are unrecoverable under a quantum break (Part I); consensus stops their circulation and drains the pool through a public turnstile (Part H).
- Bitcoin contrast: a soft fork narrowing what an output type may do — spendable only back to its own key or out of the type, no longer created by coinbase or funded from outside (Part E).

Write

$$\begin{aligned} \text{Acc}_0 &:= Q(D), & \text{Acc}_{i+1} &:= (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i, & \text{Sinsemilla}_D(M) &:= \text{Acc}_\ell, \\ \text{SinsemillaCommit}_r(D, M) &:= \text{Sinsemilla}_{D\parallel\text{-M}}(M) + [r] H_D \end{aligned}$$

M in 10-bit chunks $m_1, \dots, m_\ell$; $\dot{+}$ is incomplete addition; short forms return **Extract**.

Say

- The accumulator starts at $Q(D)$, a generator fixed by the domain D ; per chunk, add that chunk's generator, then add the accumulator again; the hash is the final accumulator.
- Incomplete addition: the plain formula, for inputs neither equal nor opposite.
- The generators $Q(D)$, $P[0], \dots, P[2^{10} - 1]$ and H_D are **GroupHash** outputs: no known discrete-log relation among them.
- Collision resistance reduces to discrete log on Pallas.

Write

- per 10-bit chunk: one table lookup + two additions
- bit-oriented hash: field arithmetic + range checks; dominates the circuit
- Bitcoin: nobody recomputes SHA-256 inside a proof

Say

- Here the spender must recompute the commitment inside a proof, so the hash's in-circuit cost is the whole question.
- Lookup and addition are native to a circuit with lookups; a bit-oriented hash must be re-expressed through field arithmetic and range checks.

The note commitment: what the chain stores

exists

Write

$$\begin{aligned}\text{cm} &:= \text{NoteCommit}_{\text{rcm}}(\mathbf{g}_d^* \parallel \mathbf{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \\ \text{cmx} &:= \text{Extract}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}}\end{aligned}$$

on chain	cmx , one field element (plus a ciphertext, Part F)
private	$d, \mathbf{pk}_d, v, \rho, \psi, \text{rcm}$: recipient, amount, seeds, trapdoor

Say

- The commitment is the Sinsemilla commitment under its own domain, in the Pallas group; the star is the 32-byte point encoding.
- The tree consumes a field element, hence **Extract**: the chain sees cmx , one field element per note.

Write

- hiding: $[rcm]H_D$ masks the hash — perfectly for a uniform rcm , computationally (PRF) for the seed-derived one
- binding: one cm opened as two notes $\Rightarrow$ a discrete-log relation
- CT: commit to the amount + a range proof beside it; here: recipient + amount, opened only inside the proof (Part E)

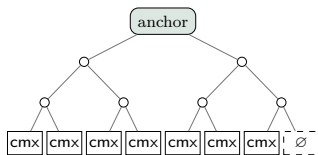
Say

Confidential Transactions commits to the amount and publishes a range proof beside it; this commitment covers recipient and amount, and is opened only inside the zero-knowledge proof (Part E).

Two structures that only grow

exists, not twice

	set	operations
Bitcoin	the UTXO set	insert on output, delete on spend
each pool	the note commitment tree	append every cmx; a root per block
	the nullifier set	insert every nf; never delete



public cmx leaves; one tree per pool

Say

- Bitcoin keeps one structure that shrinks; each pool keeps two that only grow.
- The tree expresses existence of value, as the UTXO set does; the nullifier set is the “deleted” half, and nothing maps a nullifier back to a leaf.

Write

- depth 32 $\Rightarrow$ 2^{32} leaves, creation order, one tree per pool
- node = Sinsemilla short hash of (10-bit layer tag || 255-bit child || 255-bit child)
= 520 bits = 52 chunks
- empty position = 2, the x -coordinate of no Pallas point
- Bitcoin txid tree: rebuilt per block, SHA-256d, depth follows the transaction count

Say

- One tree per pool for the pool's whole life, filled in creation order.
- The sentinel 2 is the x -coordinate of no Pallas point, so no leaf can collide with "empty".
- Bitcoin's txid tree is rebuilt per block; this tree is one, fixed in depth, and only ever extended.

Minting: what an output writes to the chain

exists, conserved

Write

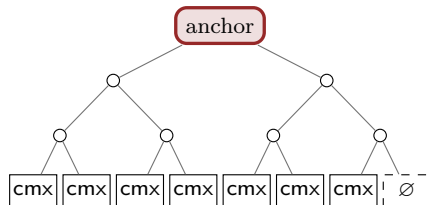
- output side writes: cmx_{new} , epk , C_{enc} (to the recipient), C_{out} (to the sender's outgoing key)
- value commitment: $\text{cv} := [v_{\text{old}} - v_{\text{new}}] V + [\text{rcv}] R$
- Action 1 mints Bob's note: $v = 1.5 \text{ ZEC} = 1.5 \cdot 10^8$ zatoshi, $\rho_{\text{new}} :=$ the nullifier of Alice's note
- on chain: cmx , cv , epk , C_{enc} , C_{out} — opaque strings

Say

- Bitcoin writes **amount** and **scriptPubKey** in clear and the UTXO set indexes them; here nothing readable about the new note is written.
- Hidden twice: inside cm , bound to the recipient; and the net value inside cv , whose homomorphism lets a validator check conservation without any amount (Part E').
- Delivery fields are opaque to the validator; only their owners open them (Part F).

Resting: the frontier and the anchor

exists



a Bitcoin input names one outpoint; this spend names a tree

Write

- append on the right $\Rightarrow$ no old root changes; an anchor names one fixed tree, for ever
- block's anchor = its tree's final root; a transaction's anchor = an *earlier* block's
- frontier = newest leaf + one sibling root per filled subtree along its path
- any main-chain block since the pool's activation: no window

Say

- An anchor names one fixed historical tree; a transaction's must be an earlier block's: no spend of an unmined note, no mempool chains.
- The frontier is enough to append and recompute the root; a wallet keeps its own notes' paths as retained nodes and subtree roots (Part F).
- No window: consensus accepts the anchor of any main-chain block since the pool's activation, so proving is decoupled from broadcast.

Which anchor a wallet picks

exists

Write

- consensus: no depth mandated; wallet policy
- deployed default: newest checkpoint at or below $H - 3$, H the target height; per pool, oldest taken
- ZIP-315: input availability; fixed depth hides trusted status (untrusted 10 confirmations, trusted 3)
- the Guide: anchor's tree = candidate set; newest admissible = largest; a deep anchor fingerprints
- old anchor $\neq$ soundness hole; second spend is the nullifier's job

Say

- A checkpoint is a tree state the wallet stored at a block boundary, one series per pool; checkpoint gaps and the cross-pool minimum can make the anchor older.
- Membership in an older, smaller tree proves the note was committed even earlier.
- Bitcoin has no such choice: an input names its outpoint, whatever the tip.

When the anchor vanishes: reorganisations

exists

Write

- reorg strikes the anchoring block $\Rightarrow$ anchor invalid; deeper is rarer, never impossible
- reorg depth = node policy: Zebra ≤ 1000 blocks; the specification describes 100
- deployed wallets: truncate to a checkpoint below the fork $\rightarrow$ rescan $\rightarrow$ rebuild; same nullifiers
- specified, wallet support pending: re-anchor with txid unchanged (ZIP-374 v2; the proof pins the anchor)

Say

- The rebuilt spend reveals the same nullifiers, because each is a fixed function of its note.
- ZIP-374 v2 makes v6 anchors deferrable, and the proof rather than the signature pins them.
- Bitcoin contrast: a reorged transaction stays valid wherever its inputs survive; here the tree it named may no longer exist.

Write

- candidate set = every leaf of the Ironwood tree under the anchor
- no participants, no coordinator, no round
- CoinJoin: set = one transaction's co-signers; the graph remains
- narrows the set: metadata — dummy or spent leaves, timing, fees, transparent components — not the proof

Say

- The candidate set of Alice's spend is everyone's notes since the pool's activation, when the tree began empty; the other leaves' owners never take part.
- In CoinJoin every output is a child of the whole input set.
- Tree size alone does not measure the set; the metadata that narrows it is Part H's.

Write

$$\begin{aligned} \text{nf} &:= \text{Extract}\big([F_{\text{nk}}(\rho) + \psi] G^{\text{nf}} + \text{cm}\big) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ G^{\text{nf}} &:= \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard}, K) \end{aligned}$$

- keyed by nk ; seeded by ρ ; salted by ψ ; bound to cm ; $F_{\text{nk}} = \text{Poseidon}$ keyed by nk
- $\rho_{\text{new}} := \text{nf}_{\text{old}}$ (the note spent in the same Action)
- dummy spend: $\rho = \text{Extract}(\text{random Pallas point})$; still publishes nf

Say

- Bitcoin's spent marker is the outpoint, the coin's name; this one names no leaf.
- Poseidon is an algebraic hash the circuit evaluates natively.
- Every note is born from a spend and inherits its nullifier as ρ ; a dummy spend is an Action with no real input.

Uniqueness and unlinkability, separately

not twice

Write

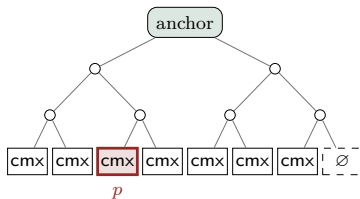
- duplicate nf rejected per pool $\Rightarrow$ accepted ρ unique within the pool
- leaf index never enters nf (private witness of membership only)
- uniqueness: ingredients fixed at creation; two notes, one nf $\Rightarrow$ discrete-log relation on Pallas
- unlinkability: without nk the masking term looks random — DDH on Pallas, or Poseidon as a PRF

Say

- Duplicate nullifiers are rejected per pool, so forged notes cannot be made to share a nullifier; position is a private witness of the membership check, not an input to nf.
- Either assumption suffices for unlinkability: defence in depth.
- Bitcoin's marker is unique because it is the coin's name; this one because two names would break discrete log.

Alice's leaf: the position is a secret

exists, not twice



position p and the path stay inside the proof

Write

- private witness: p and its 32 siblings; public: the anchor
- nf fixed at mint: nk , ρ , ψ , cm all set; only a holder of nk computes it
- node at spend: $\text{nf} \notin \text{set} \Rightarrow \text{record nf}$
- SPV's Merkle path to a public root, run inside the zero-knowledge proof

Say

- Alice's 2 ZEC note sits at position p , recorded by her wallet when trial decryption succeeded.
- At spend she reveals nf ; the node checks it is not in the set, then records it. Nothing links nf to the leaf at p .
- The SPV proof — a Merkle path to a public root — runs here inside the zero-knowledge proof.

Ask

Alice's restored wallet has found its old notes. How does it learn which are spent?

Answer

- The chain never marks a leaf spent, so the tree cannot say.
- With nk the wallet computes each note's nullifier and discards those already in the pool's nullifier set; compact blocks carry every nf for this, and to supply ρ when detecting the note minted beside it (Part F).
- With the incoming keys alone it sees notes arrive but cannot tell which have left: spent status is a capability of nk (Part D).
- Bitcoin: a restored wallet asks the UTXO set.

Part D — Keys and addresses

Write

1. one seed: 32–252 bytes, at least 256 bits of entropy
2. $(\text{sk}_i, c_i) := \text{halves of PRFexpand}_{c_{\text{par}}}(\text{0x81} \parallel \text{sk}_{\text{par}} \parallel i), \quad i \geq 2^{31}$
3. transparent: $m/44'/133'/\text{account}'/\text{change}/\text{index}$ — two lowest levels non-hardened, xpub at the account
4. Orchard: $m_{\text{Orchard}}/32'/133'/\text{account}'$ — hardened only, chain code discarded at the account, no change level

Say

- One seed roots both trees, as one BIP-32 seed roots a Bitcoin wallet: one backup recovers every key.
- Orchard has no xpub mode (a public key deriving child public keys), so no Orchard extended full viewing key exists.
- No change level: a shielded change output is indistinguishable on chain from any other output, so a change subtree would separate nothing.

Two scopes per account: external and internal

authorised

Write

1. $r_{ivk}^{int} := \text{ToScalar}(\text{PRFexpand}_{r_{ivk}}(0x83 \parallel ak \parallel nk))$
2. $fvk^{int} := (ak, nk, r_{ivk}^{int})$
3. same ask spends, same nk nullifies; ivk, dk, ovk re-derived
4. $fvk \rightarrow fvk^{int}$; $(dk, ivk) \rightarrow$ external scope only

Say

- Only r_{ivk} changes. The map `ToScalar` reduces the 512-bit PRF output modulo the Pallas scalar-field order; an internal address does not decrypt under the external ivk.
- A full viewing key sees both scopes. An external (dk, ivk) — a payment processor — sees the external scope only: neither the internal scope nor any ovk follows from it.
- BIP-44 separates change by a path level on a public graph; here the separation is a second full viewing key, and the change output looks like any other output on chain.

Write

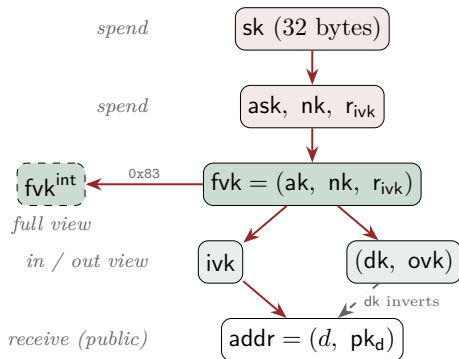
1. internal addresses $\leftarrow$ change, auto-shielded funds
2. shielding = moving transparent funds into the pool (automatic, to an internal address)
3. Alice's change: 0.4999 ZEC $\rightarrow$ internal address, index 0

Say

- What BIP-44 puts in the path sits here in the key.
- The deployed wallet library sends change to the internal address at index 0.
- A Bitcoin change output sits on the change branch of a public graph; here the change output is indistinguishable from any other, and only the internal scope's keys find it.

The ladder: one secret, graded capabilities

authorised / not twice



$ask, nk, r_{ivk} := \text{PRFexpand}_{sk}(0x06/0x07/0x08)$

$ak := [ask] G^{\text{SpendAuth}}; nk, r_{ivk} \text{ carried}$

$ivk := \text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk)$

$(dk, ovk) := \text{halves of PRFexpand}_{r_{ivk}}(0x82 \parallel ak \parallel nk)$

$d := \text{FF1-AES256}_{dk}(j), g_d := \text{GroupHash}(d),$

$pk_d := [ivk] g_d$

Write

1. $\text{ask}, \text{nk}, r_{\text{ivk}} := \text{PRFexpand}_{\text{sk}}(0\text{x}06/0\text{x}07/0\text{x}08)$
2. $\text{ak} := [\text{ask}] G^{\text{SpendAuth}}; \text{nk}, r_{\text{ivk}}$ carried
3. $\text{ivk} := \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$
4. $(\text{dk}, \text{ovk}) := \text{halves of PRFexpand}_{r_{\text{ivk}}}(0\text{x}82 \parallel \text{ak} \parallel \text{nk})$
5. $d := \text{FF1-AES256}_{\text{dk}}(j), \text{g}_d := \text{GroupHash}(d), \text{pk}_d := [\text{ivk}] \text{g}_d$

Say

- Red arrows are one-way: a rung is handed down without the powers above it.
- Bitcoin's BIP-32 has two rungs, xprv and xpub , and the public ledger supplies the rest; here every rung between is a separate secret, and only the address is public.
- The pair (dk, ovk) comes from fvk by a PRF, not from ivk : the two branches leaving fvk are siblings.
- The circuit re-walks $\text{fvk} \rightarrow \text{ivk} \rightarrow \text{pk}_d$ (A7) with g_d as a witness; the step $j \rightarrow d \rightarrow \text{g}_d$ is never checked in the circuit (the dashed grey edge).

What holds the rungs apart

authorised / not twice

step	operation	why it is one-way
$sk \rightarrow (ask, nk, r_{ivk})$	$\text{PRFexpand}_{sk}(0x06/0x07/0x08)$	PRF (BLAKE2b)
$ask \rightarrow ak$	$[ask] G^{\text{SpendAuth}}$	discrete log on Pallas
$fvk \rightarrow ivk$	$\text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk)$	hiding under r_{ivk}
$fvk \rightarrow (dk, ovk)$	halves of $\text{PRFexpand}_{r_{ivk}}(0x82 \parallel ak \parallel nk)$	PRF
$fvk \rightarrow fvk^{\text{int}}$	$\text{PRFexpand}_{r_{ivk}}(0x83 \parallel ak \parallel nk)$	PRF
$ivk \rightarrow pk_d$	$[ivk] g_d$	discrete log on Pallas
$j \rightarrow d$	$\text{FF1-AES256}_{dk}(j)$	<i>not</i> one-way: dk inverts it

Write

1. the scalar ivk : a commitment, not a hash — binds *and* hides (ak, nk)
2. one r_{ivk} keys both Commit^{ivk} and the 0x82 PRF $\Rightarrow$ joint assumption once dk or ovk is exposed
3. a separate nk : spent status without spend authority

Say

- Three kinds of wall: a PRF (BLAKE2b) at the top and at the 0x82 and 0x83 splits; a discrete logarithm on Pallas at $ask \rightarrow ak$ and $ivk \rightarrow pk_d$; a hiding commitment at $fvk \rightarrow ivk$.
- An ivk holder learns neither ak nor nk . The viewing key holds nk , never ask .
- The last row is deliberately not one-way: the dk holder maps any of its addresses back to an index. Bitcoin has one wall, $xprv$ to $xpub$; the rest of its ladder is public.

Graded capabilities: spend, full view, spent status

authorised / not twice

capability	holds	can	cannot
spend	ask (from sk)	authorise spends, both scopes	—
full view	$\text{fvk} = (\text{ak}, \text{nk}, r_{\text{ivk}})$	everything below, both scopes; every address and its index	spend: ask is a discrete log away
spent status	nk (inside fvk)	the nullifier of a note it holds	spend; find notes (needs ivk)

Write

1. Bitcoin: only the spend row is secret (xprv)

Say

- Each row holds strictly less than the one above. A full viewing key sees everything and moves nothing: ask is a discrete logarithm away from ak.
- Full view and spent status are what Bitcoin's public ledger shows to everyone.

Graded capabilities: incoming, outgoing, receive

exists

capability	holds	can	cannot
incoming view	(dk, ivk)	detect and decrypt incoming; derive all 2^{88} addresses and invert them to indices	nullifiers, so spent status; outgoing; the internal scope
outgoing view	ovk	open C_{out} of what this account and scope sent (a sender may set $ovk = \perp$: then no ovk opens it)	incoming; the internal ovk
receive	(d, pk_d)	be paid	anything else — the only public item

Write

1. every row but the last is a distinct secret

Say

- Bitcoin's ledger collapses everything between $xprv$ and the address into what anyone can read.
- The incoming view sees arrivals, not departures: no nullifiers, so no spent status, no outgoing view, no internal scope.
- The outgoing view may open nothing: a sender may set $ovk = \perp$.
- The address is the only public item: it lets one be paid and does nothing else.

Viewing keys: xpub is the bridge, not the analogue

exists / not twice

Write

1. xpub = watch-only: enumerates addresses; the public ledger supplies incoming, outgoing, spent status — to anyone
2. (dk, ivk) = watch-only for incoming: enumerates *and* decrypts
3. three differences: (1) nothing visible without the key; (2) spent status = nk; (3) outgoing = ovk
4. “viewing” = a capability, not publicity

Say

- Nothing is visible without the key because the outputs are ciphertext. The key nk computes the nullifiers matched against the chain’s set; the key ovk opens C_{out} . The xpub gets both free only because Bitcoin’s ledger is public.
- Every viewing key is secret material disclosed deliberately — to an auditor, a processor — and validation needs none of them.

Diversified addresses: one key, 2^{88} addresses

exists

Write

1. $d := \text{FF1-AES256}_{\text{dk}}(j)$
2. $\mathbf{g}_d := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d)$
3. $\text{pk}_d := [\text{ivk}] \mathbf{g}_d, \quad \text{addr} := (d, \text{pk}_d)$
4. $2^{88} \approx 3.1 \times 10^{26}$ addresses per key; raw receiver $11 + 32 = 43$ bytes
5. random 88-bit draws collide near 2^{44} ; FF1 never collides, and dk inverts it

Say

- Every index works: GroupHash is total (a fixed fallback covers the identity), so there is no rejection loop. All addresses are spent by one ask and found by one ivk , at no extra scanning cost.
- A keyed permutation, not a counter and not a hash: the wallet stores nothing per address.
- Wallet-side only, never in the circuit: the circuit takes $\mathbf{g}_d$ as a witness (the ladder board) and never sees j , so AES is fine here.

Unlinkable, even to the payer

exists

Write

1. (d, pk_d) and $(d', \text{pk}_{d'})$: $\text{pk}_d = [\text{ivk}] g_d$, $\text{pk}_{d'} = [\text{ivk}] g_{d'}$ — same ivk ?
2. deciding it = DDH on Pallas, **GroupHash** a random oracle
3. only (dk, ivk) can check: re-derive at the decrypted index
4. never a chosen-diversifier oracle

Say

- Given two addresses of different owners and a third derived from one of them, no one can tell which. Alice, who paid one of Bob's addresses, learns nothing about the other.
- A chosen-diversifier oracle — an address for an attacker's d under your ivk — breaks unlinkability for every address of that ivk .
- Under one xpub , addresses link on the public graph once spent together (Part A), and a leaked xpub links all beneath it, spent or not. Here an address reveals only itself.

Unified addresses (ZIP-316): one string, several receivers

exists

Write

1. typecodes: 0x00 P2PKH, 0x01 P2SH, 0x02 Sapling, 0x03 Orchard — at most one each, at least one shielded
2. ascending typecode $\rightarrow$ F4Jumble (whole payload, HRP included) $\rightarrow$ Bech32m
3. sender: Orchard $>$ Sapling $>$ transparent; may refuse a type, never reorder
4. Orchard receiver $\rightarrow$ the Ironwood pool

Say

- F4Jumble is an unkeyed four-round Feistel permutation of the whole payload, so a substituted receiver changes the whole string, not one region of it.
- An Orchard receiver names the protocol, not a pool; all new value enters the Ironwood pool, so a payment to it lands in the Ironwood tree.
- The same container with viewing-key items is a unified full or incoming viewing key.
- One Bitcoin address encodes one script type; here one address lets the sender's wallet choose the pool.

Bob's address in the example

exists

Write

1. $\text{addr}_B = (d_B, \text{pk}_d)$, 43 bytes
2. $\text{g}_d := \text{GroupHash}^{\text{Pallas}}(\text{z.cash} : \text{Orchard-gd}, d_B)$, $\text{epk} := [\text{esk}] \text{g}_d$
3. $\mathbf{n} := (\text{addr}_B, 1.5 \text{ ZEC}, \rho, \psi, \text{rcm})$
4. on chain: inside cm , inside C_{enc} , as epk under a fresh esk ; lead byte 0x03
5. change 0.4999 ZEC $\rightarrow$ Alice's internal address, index 0

Say

- Bob hands Alice a unified address; her wallet takes its Orchard receiver and learns nothing else: no ivk , nk or ak , no index j , no link to Bob's other addresses.
- The note is minted in the Ironwood tree; Bob's one ivk trial-decrypts both pools. No address of Alice's appears either.
- Bitcoin displays the script beside the amount; here neither appears in the clear.

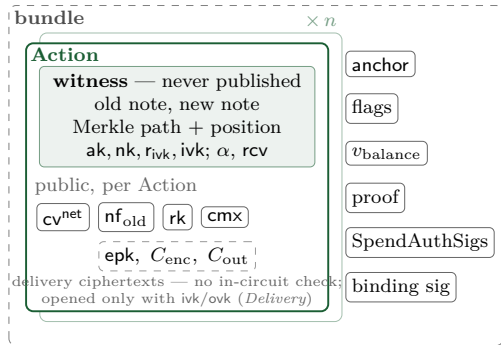
Ask

Which key does an accountant need to audit a company's incoming *and* outgoing shielded payments — and what can they not do with it?

Answer

- One unified full viewing key per account. Inside it, fvk yields ivk (incoming), nk (which of those notes are spent) and ovk (outgoing) — for the external scope and, through the $0x83$ step, the internal one, so change is covered too.
- Not spend: ask is a discrete logarithm away from ak . Not wander into a sibling account: only hardened derivation exists, and no Orchard extended full viewing key.
- Not read what was sent under some other ovk , or under none ($ovk = \perp$); nor anything received by an account they were not given.

Part E — The Action statement



Write

- one Action = one old note spent + one new note created
- witness (never published): old note, new note, path + position, $ak, nk, r_{ivk}, ivk, \alpha, rcv$
- rim, per Action: $cv^{net}, nf_{old}, rk, cmx$ — the four the proof binds
- dashed: epk, C_{enc}, C_{out} — ciphertexts, read by no clause

Say

- The shape is fixed: an Action never shows which it “really” is.
- Bitcoin: an input names its coin and shows its script; an output shows its amount and its script. Here every field is a commitment, a serial number, a disguised key, or a ciphertext.

The bundle around it

all four

per bundle	once	role
anchor	one root, shared by every Action	the tree the spends prove membership in
flags	enableSpends, enableOutputs, enableCrossAddress	consensus gates (A8–A10)
v_{balance}	signed, cleartext	net value leaving the pool
proof	one aggregate Halo 2 proof	covers all n Actions at once
binding sig	one signature	proves the values balance
per Action	SpendAuthSig under rk	the owner authorised this spend

Write

- one Action = 820 public bytes, 660 of them ciphertext
- Alice: 2 Actions, one 7264-byte proof
- node checks: proof, anchor, nullifier set, two signatures — never a note

Why a spend and an output are fused

all four

Write

$$n_{\text{act}}(s, o) = \max(\max(s, o), 2) \quad \text{Alice: } s = 1, o = 2 \Rightarrow n_{\text{act}} = 2$$

- spend list and output list each padded to n_{act} ; floor 2
- Alice: 1 spend, 2 outputs (Bob, change) $\Rightarrow$ one dummy spend

Say

- A Bitcoin transaction shows its true input and output counts. Here the count is n_{act} , and the fee is computed from that public count (*The transaction and the turnstile*).
- Every Action looks like every other: one spend slot, one output slot, whatever the payment needed.
- This count and the shuffle are the Ironwood component's (cross-address enabled). A sealed Orchard-pool bundle pads to $s + o$ and pairs each spend with an output to the spent note's own receiver.

Write

- dummy spend: value 0, fresh random spending key, $\rho = \text{Extract}(\text{random Pallas point})$; still publishes a nullifier
- dummy output: value 0, fresh random address, $\rho := \text{nf}_{\text{old}}$ like any output's
- shuffle spends and outputs *independently*, then zip
- Alice: spend side {2 ZEC note, dummy}; output side {Bob, change}

Say

- Which spend shares an Action with which output is the shuffle's choice.
- Bitcoin: an input is an input and an output is an output. Here a dummy spend looks exactly like a real one and, like a real one, publishes a nullifier the pool records.

Write

- *“Some leaf under this anchor is a note I own; this is its serial number; this commitment hides its value change; this signing key is a fresh disguise of the owner's — and I have shown you neither the leaf, the note, nor the owner's key.”*
- public: anchor , cv^{net} , nf_{old} , rk , cmx , three flags — 8 names, 10 field elements
- witness: both notes, path + position, ak , nk , r_{ivk} , ivk , α , rcv
- relation: ten clauses A1–A10; one verifying key, selected by branch; a proof of nine fails

Say

- The two Pallas points enter by their coordinates. Here cv^{net} and nf_{old} are the data model's cv and nf , marked *net* (this Action's value change) and *old* (the note spent).
- A proof of nine clauses is a proof for a different circuit, and it fails.
- Bitcoin's validator runs *your* script. This validator runs one verifier keyed to one fixed circuit; the transaction supplies inputs, never logic.

The ten clauses, ordered by the four checks

all four

Bitcoin's check	clause	what the proof establishes
coin exists (UTXO lookup)	A3	a Merkle path from cm_{old} to the anchor
not spent twice (entry deleted)	A5	nf_{old} is this note's serial, keyed by nk
conserved ($\sum \text{in} \geq \sum \text{out}$)	A4	cv^{net} hides $v_{\text{old}} - v_{\text{new}}$, both below 2^{64}
authorised (script + signature)	A6, A7	rk disguises ak ; ak, nk sit behind the address
well-formed	A1, A2	both commitments open to notes; $\rho_{\text{new}} = \text{nf}_{\text{old}}$
gated	A8, A9	spends and outputs switched on by the flags
sealed	A10	cross-address transfers switched off, or not

Write

- 4 Bitcoin checks $\rightarrow$ 7 clauses (A1–A7); 3 consensus gates (A8–A10)
- stays with the node: “has this serial been seen?”

Alice's two Actions against the ten clauses

all four

clause	Action 1: 2 ZEC $\rightarrow$ Bob 1.5	Action 2: dummy $\rightarrow$ change 0.4999
A1	Alice's 2 ZEC note opens cm_{old}	a zero-value dummy opens cm_{old}
A2	Bob's note; $\rho_{\text{new}} := \text{nf}$ of Alice's note	change note; $\rho_{\text{new}} := \text{nf}$ of the dummy
A3	a 32-sibling path to the Ironwood anchor	gated off: $v_{\text{old}} = 0$
A4	cv^{net} commits to +0.5 ZEC	cv^{net} commits to -0.4999 ZEC
A5	nf of Alice's note, under her nk	nf of the dummy, under its random nk
A6	$\text{rk} = \text{ak} + [\alpha] G^{\text{SpendAuth}}$, Alice's ak	the same, the dummy's ak
A7	Alice's ak , nk sit behind her address	the dummy's key, behind its address
A8, A9	both flags set: both products vanish	the same
A10	$\delta = 0$ (Ironwood): nothing constrained	the same

Write

$$+0.5 - 0.4999 = +0.0001 \text{ ZEC} = 10\,000 \text{ zatoshi} = \text{fee} = v_{\text{balance}}$$

What Alice's public fields carry

all four

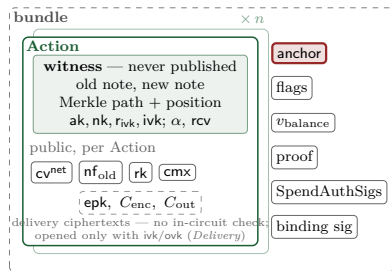
field	Action 1	Action 2
cv^{net}	+0.5 ZEC under a fresh rcv	-0.4999 ZEC under a fresh rcv
nf_{old}	Alice's note's serial: the note is retired	the dummy's serial, recorded all the same
rk	a fresh disguise of Alice's ak	a fresh disguise of the dummy's ak
cmx	Bob's 1.5 ZEC note	Alice's 0.4999 ZEC change note, at her internal address
epk, C_{enc}	to Bob's pk_d ; opened with his ivk	to Alice's internal pk_d ; found with ivk^{int}
C_{out}	under Alice's ovk: she can re-read it	no ovk for change (wallet default): unrecoverable
bundle	one Ironwood anchor; flags (1, 1, 1); $v_{\text{balance}} = +10\,000$ zatoshi; one proof; two SpendAuthSigs; one binding signature	

Write

- which is “Action 1” on the wire: the shuffle decides
- 2 nullifiers, 2 commitments: every two-Action bundle

Coin exists — A3, the path to the anchor

exists



Ask

Which of the four checks?

Write

- fold $\text{Extract}(cm_{old})$ up 32 witnessed siblings to root
- $v_{old} \cdot (\text{root} - \text{anchor}) = 0$
- witness: siblings, position bits; public: anchor, one per bundle

Say

- Bitcoin looks the coin up in the UTXO set, and so names it. This clause says only “some leaf under this anchor”: the candidate set is the Ironwood tree under the anchor.
- A zero-value dummy skips the check by arithmetic, not by a flag.

Answer

Coin exists.

No double spend — A5, the nullifier

not twice

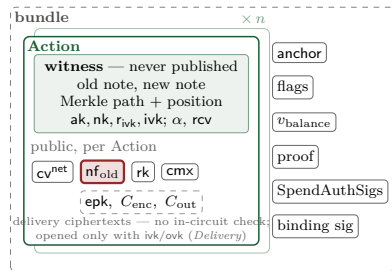
Write

$$nf_{old} = \text{Extract}([\text{Poseidon}_{nk}(\rho_{old}) + \psi_{old}] G^{nf} + cm_{old})$$

- three lines: membership (A3), serial (A5), balance (A4)

Say

- Bitcoin deletes the entry; here an unlinkable serial is added.
- Clause A5 binds the published serial to the spent note under the witnessed nk ; otherwise a spender could invent a fresh serial per spend and the set would never repeat.
- “Seen before?” is global history: the node checks the set, the proof cannot.



Ask

Which check?

Answer

Not spent twice.

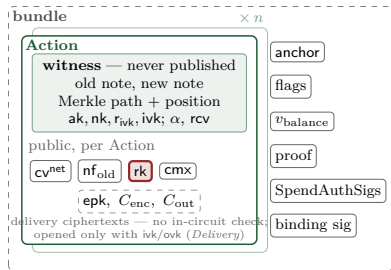
Write

$$\text{ivk} = \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk}), \quad \text{pk}_d = [\text{ivk}] \text{g}_d \quad \text{for every } d$$

- one ak per wallet, not per coin
- publish ak + signature $\Rightarrow$ every spend, from every address, linked
- needed: a fresh key per spend that still proves the same secret

Say

- Every diversified address shares ak through ivk (*Keys and addresses*).
- Bitcoin's version of this is address reuse, and it is optional. Here it would be structural.



Write

$$rk = ak + [\alpha] G^{SpendAuth}, \quad rsk = ask + \alpha$$

- circuit checks the shift; node checks a signature under rk
- uniform $\alpha \Rightarrow$ uniform rk , whatever ak

Say

- Clause A6: the published key is ak shifted by a fresh, secret α .
- Bitcoin bridge: Schnorr with a per-spend tweak — a non-hardened BIP-32 child whose tweak is uniform and never published.

Ask

Which check?

Answer

Authorised.

Write

$$\text{ivk} = \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk}), \quad \text{pk}_{\text{dold}} = [\text{ivk}] \text{g}_{\text{dold}}$$

- one identity: signs (A6), nullifies (A5), is paid (A7)
- circuit: the key is the owner's; signature under rk: the spender holds it

Say

- Clause A6 alone proves a disguise of some ak ; clause A7 ties that ak , and the nk of A5, to the spent note's address by the ladder of *Keys and addresses*.
- Bitcoin: a P2PKH address *is* the key's hash, so “this key matches this coin” is one hash comparison in the clear. Here the address is a commitment to the key, and the match is proved.

Write

$$\text{cm}_{\text{old}} = \text{NoteCommit}_{\text{rcm}_{\text{old}}}(\mathbf{n}_{\text{old}}), \quad \text{Extract}(\text{NoteCommit}_{\text{rcm}_{\text{new}}}(\mathbf{n}_{\text{new}})) = \text{cmx},$$

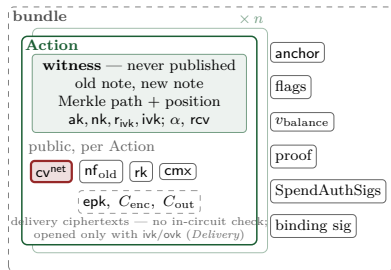
with $\rho_{\text{new}} := \text{nf}_{\text{old}}$.

- note = address, value, ρ , ψ ; commitment binding $\Rightarrow$ one leaf, one note
- chain stores only cmx

Say

- A Bitcoin output is plaintext: well-formed means it parses. Here the note is hidden, so “this commitment opens to a note” must be proved.
- The chain $\rho_{\text{new}} := \text{nf}_{\text{old}}$ is enforced here: accepted seeds are unique within the pool, and two notes cannot share a nullifier.

Write



Ask

Which check?

$$cv^{net} = [v_{old} - v_{new}] V + [rcv] R,$$

$$v_{old}, v_{new} \in [0, 2^{64}), \quad sign \in \{-1, +1\}$$

- group order $\approx 2^{254}$: a value near the order is a small negative one

Say

- Bitcoin checks $\sum in \geq \sum out$ on amounts it can read; here they are hidden. Clause A4 commits to this Action's value *change* under a fresh trapdoor rcv ; the bundle sums these (*Balance and authority*).
- Unbounded, the books would “balance” while minting.
- Elements' range proof exists for the same reason; here it lives inside the Action proof, with the opening.

Answer

Conserved.

Write

$$v_{\text{old}} \cdot (1 - \text{enableSpends}) = 0, \quad v_{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$$

- flag clear $\Rightarrow$ only value 0 passes
- coinbase: `enableSpends` = 0; Alice: both flags set

Say

- Bitcoin's coinbase has one null input by format. Here the same rule is arithmetic on a hidden value — the only way to gate what the node cannot read.
- The flags are bundle-level bits the node supplies as public inputs; the circuit only multiplies. A cleared `enableSpends` admits dummies and nothing else.
- A coinbase must clear `enableSpends`: its Actions spend dummies and create the block's shielded reward.

Write

$$\begin{aligned}\delta \cdot (x(\mathbf{g}_{\text{dold}}) - x(\mathbf{g}_{\text{dnew}})) &= 0, & \delta \cdot (y(\mathbf{g}_{\text{dold}}) - y(\mathbf{g}_{\text{dnew}})) &= 0, \\ \delta \cdot (x(\mathbf{pk}_{\text{dold}}) - x(\mathbf{pk}_{\text{dnew}})) &= 0, & \delta \cdot (y(\mathbf{pk}_{\text{dold}}) - y(\mathbf{pk}_{\text{dnew}})) &= 0\end{aligned}$$

- $\delta := \text{disableCrossAddress}$, the tenth public input; on the wire negated as `enableCrossAddress`, bit 2 of the flags byte
- Ironwood 0b111: $\delta = 0$ (Alice); Orchard pool 0b011: $\delta = 1$

Say

- With $\delta = 1$ the four products force the new note to pay the spent note's own receiver, as full points. With $\delta = 0$ they vanish.
- A Bitcoin output type cannot be told “pay only yourself”; this one can.

The seal: three rules, and two checks that are not clauses

sealed

Write

- **No issuance** — coinbase: empty Orchard component
- **No circulation** — every Orchard-pool Action: `enableCrossAddress = 0`
- **No entry** — per transaction: $v_{\text{balance}}^{\text{Orchard}} \geq 0$
- not clauses: canonical encodings; $\text{rk} \neq \text{identity}$

Say

- No circulation is enforced by the verifying key consensus selects by branch, so it binds whatever the transaction version.
- A sealed-pool note pays anyone by *leaving*: the value exits through a positive v_{balance} , and the output slot holds a zero-value note to the spent note's own receiver.
- A Bitcoin soft fork narrows what a script may do, in the clear; this seal narrows what a hidden note may do, by a public flag the circuit reads.

Ask

Which clause would you delete to counterfeit, and what stops you?

Answer

- Delete A3 and spend a note nobody minted; delete A4's range and wrap a subtraction into new value; delete A1 and reopen one leaf as two notes; delete A5 and spend one note under a fresh serial each time.
- What stops you: the circuit is fixed, and consensus selects its verifying key by branch. A proof of a weaker statement is a proof for a different circuit, and no node holds that key.

Part E' — Balance and authority, on the running example

Write

1. Commitment: $C(v; r) := [v] V + [r] R$; nobody knows $\log_V R$.
2. Toy group: $\mathbb{Z}_q$, $q = 65\,537$, $V = 7$, $R = 11$; one unit = 10^4 zatoshi = the fee.

note	v (units)	r	$C(v; r)$
Alice's, 2 ZEC	20 000	4 321	56 457
Bob's, 1.5 ZEC	15 000	1 234	53 037
change, 0.4999 ZEC	4 999	9 876	12 555

3. Cancellation: $C_A - C_B - C_c - [1] V = 56\,395 = [4\,321 - 1\,234 - 9\,876] R = [58\,748] R$.

Say

- Bitcoin's node adds up amounts it can read; here the amounts sit inside commitments. Logs are free in the toy; on Pallas they are not.
- What survives is a commitment to zero under a net randomness only the author who drew the three r knows; the next boards turn that into a signature.

Write

1. Bitcoin: three amounts, read and subtracted.
2. Confidential Transactions: one commitment per input and per output; the two sums differ by the fee.
3. Here, one commitment per Action: $\mathbf{cv} := [v_{\text{old}} - v_{\text{new}}] V + [\mathbf{rcv}] R$.
4. Alice: two envelopes for three notes, plus v_{balance} beside them.

Say

- Confidential Transactions keep Bitcoin's shape: a commitment per input and per output.
- Here the unit is the Action, so the commitment is already a net: spend minus output, under a fresh trapdoor $\mathbf{rcv}$.

Write

1. Wrap-around: $C(q + 3; 5) = C(3; 5) = 76$; q units = 6.5537 ZEC.
2. Counterfeit change: $4\,999 + q = 70\,536$ units = 7.0536 ZEC from a 2 ZEC note.
3. Same envelope: $C(70\,536; 9\,876) = 12\,555 = C(4\,999; 9\,876)$.
4. Toy cancellation accepts it: $20\,000 - 15\,000 - 70\,536 - 1 = -65\,537 \equiv 0 \pmod{q}$.
5. A4: $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$, $\text{sign} \in \{-1, +1\}$; Pallas order: 255 bits.

Say

- The sum cancels modulo the group order, and a value at the order commits like zero: the toy accepts a counterfeit.
- A4 closes the gap inside the Action proof: every net value is a small signed integer, and a bundle's bounded sum of bounded values cannot reach the 255-bit order.
- Elements attaches a range proof to each output; here the range check is one clause of the Action statement. Bitcoin reads each amount and bounds it.

The binding signature: a key only a balanced author knows conserved

Write

1. Per Action (A4): $\mathbf{cv} = [v_{\text{old}} - v_{\text{new}}] V + [\text{rcv}] R$.
2. Sum over the bundle: $\sum_i \mathbf{cv}_i = [\sum_i (v_{\text{old}} - v_{\text{new}})_i] V + [\mathbf{bsk}] R$, $\mathbf{bsk} := \sum_i \text{rcv}_i$.
3. Subtract the declared flow: $\mathbf{bvk} := \sum_i \mathbf{cv}_i - [v_{\text{balance}}] V$.
4. Balanced $\iff \mathbf{bvk} = [\mathbf{bsk}] R$.

Say

- The key $\mathbf{bvk}$ is public: every node recomputes it from the wire. Its secret $\mathbf{bsk}$ is a sum of trapdoors only the author drew, and a discrete log of $\mathbf{bvk}$ only when the books balance; otherwise finding one means finding $\log_R V$.
- The binding signature is RedPallas under $\mathbf{bvk}$ on base R : a Schnorr proof of knowledge of $\mathbf{bsk}$, hence of balance, revealing no amount and not $\mathbf{bsk}$ itself.
- Bitcoin's check is arithmetic on amounts it reads; this one is a signature check under a key derived from commitments.

Write

1. Field v_{balance} : cleartext, signed, one per pool; positive = value leaving the pool.
2. Fully shielded: $\sum_i (v_{\text{old}} - v_{\text{new}})_i = \text{fee}$, so $v_{\text{balance}} = \text{fee}$.
3. Bitcoin's implicit $\sum \text{in} - \sum \text{out} = \text{fee}$ becomes a public, signed number.
4. Alice: $v_{\text{balance}} = +10\,000$ zatoshi.
5. Transparent outputs: paid from a positive v_{balance} . Shielding: a negative one.

Say

- The transparent leg enters through the same field; shielding's Actions spend dummies and create real notes. The crossing amount is public; the receiving note is not.
- Bitcoin's fee is reconstructed by looking up every input. Fully shielded, v_{balance} is the one amount the transaction states, and it is the fee; with a transparent leg the fee is again a difference of public amounts.

Authorisation on the wire: two signatures, one scheme

authorised

Write

1. RedPallas: Schnorr on Pallas, base point P a parameter.
2. Key: $X = [x] P$.
3. Signature: (N, s) , $32 + 32 = 64$ bytes.
4. Verifies iff $[s] P = N + [c] X$, $c = H(N \parallel X \parallel m)$.
5. Note: N is Part B's R , renamed; R stays the commitment base.

	per	base P and key X	knowing x means
SpendAuthSig	Action	$G^{\text{SpendAuth}}, \text{rk} = \text{ak} + [\alpha] G^{\text{SpendAuth}}$	authority: $\text{rsk} = \text{ask} + \alpha$
binding signature	pool	$R, \text{bvk} = \sum \text{cv} - [v_{\text{balance}}] V$	balance: $\text{bsk} = \sum \text{rcv}$

Say

Both signatures in the bundle are one scheme: two instantiations, two bases, two secrets.

Write

1. Circuit: rk is a shift of the ak welded into the spent note's address ($A6, A7$) — hidden pedigree.
2. Node: `SpendAuthSig` verifies under rk — public approval.
3. Signing needs $ask + \alpha$: both, or nothing.
4. One ak behind every address of the wallet $\Rightarrow$ a fresh rk per spend.

Say

- The circuit checks the key's pedigree, which must stay hidden; the node checks the signature, which is public.
- Bitcoin uses the coin's own key, revealed at its one spend; reuse links only if the address was reused. Here one ak stands behind every address of the wallet, so an un-randomised spend would publish it at every note the wallet ever spends.
- Hence a fresh, uniformly distributed rk per spend, unlinkable to ak and to every other rk .

Write

1. Message m = the ZIP-244 signature digest: a tree of personalised BLAKE2b hashes, one node per transaction component.
2. Covers all effecting data (everything under the txid): per Action nf, cmx, cv, rk, ciphertexts; the flags; v_{balance} ; the branch identifier.
3. Sighash type fixed at ALL: no ANYONECANPAY, SINGLE, NONE.
4. From v6 the anchor is authorizing data: pinned by the proof (A3), committed by the block (Part H).
5. No transparent inputs $\Rightarrow$ signature digest = txid.

Say

- No Action can be altered or transplanted into another transaction.
- Re-anchoring keeps the txid: specified (ZIP-374 v2), wallet support pending.
- Bitcoin's sighash byte chooses coverage per input; so do Zcash's transparent inputs, but every shielded signature covers all of it.

Alice's two Actions

conserved, authorised

	Action 1	Action 2
spent note, v_{old}	Alice's, 2 ZEC	dummy, 0: A3 gated off
created note, v_{new}	Bob's, 1.5 ZEC	change, 0.4999 ZEC, internal address
$v_{\text{old}} - v_{\text{new}}$	+0.5 ZEC	-0.4999 ZEC
nf	Alice's note's	the dummy's
rk, SpendAuthSig	$\mathbf{ak} + [\alpha_1] G^{\text{SpendAuth}}$, Alice signs	a throwaway key's, it signs
toy cv	$C(+5\,000; 3\,141) = 4\,014$	$C(-4\,999; 2\,718) = 60\,442$

Write

1. Action count: $n_{\text{act}} = \max(\max(1, 2), 2) = 2$.
2. Fee: $5\,000 \cdot \max(2, 2) = 10\,000$ zatoshi = v_{balance} .

Say

- Two outputs force two Actions; the dummy spend is a zero-value note under a fresh random spending key, its ρ the Extract of a random point.

The bundle balances, however it is shuffled

conserved

Write

1. Pairing shown: $\sum cv = 4\,014 + 60\,442 = 64\,456$.
2. Key: $bvk = 64\,456 - [1]V = 64\,449 = [5\,859]R$.
3. Secret: $bsk = 3\,141 + 2\,718 = 5\,859$.
4. Other pairing: $cv = 8\,484$ and $55\,972$; same $\sum cv = 64\,456$, same bvk , same bsk .

Say

- Balance is a property of the bundle, not of the pairing.
- Spends and outputs are padded, shuffled independently, and zipped; each Action draws its own rcv and α , the dummy included, so nothing on the wire says which Action is the dummy.
- Bitcoin: one input naming the 2-coin output, two outputs, and the change tell. Here: two Actions of one shape and a fee.

Ask

Checkpoint: what does the validator learn from Alice's bundle?

Answer

- The four checks pass: the anchor is a main-chain root and the proof verifies (exists); two nullifiers are absent from the set (not twice); two SpendAuthSigs verify under rk_1, rk_2 (authorised); the binding signature verifies under the recomputed bvk (conserved).
- Metadata: two Actions; $v_{\text{balance}} = +10\,000$ zatoshi, hence the fee; the anchor; two nullifiers, two cmx , two ciphertext pairs; 1 640 bytes of Actions, a 7 264-byte proof, three 64-byte signatures; when it arrived.
- Not: who, to whom, the amounts, which leaf, or which Action is the dummy — or whether either is.

Part G — The proof system

Write

1. Relation R : public statement x , private witness w .
 $\text{Prove}(x, w) \rightarrow \pi, \quad \text{Verify}(x, \pi) \in \{0, 1\}$
2. Knowledge sound: an extractor pulls a w with $(x, w) \in R$ out of any convincing prover.
3. Zero knowledge: a simulator given only x reproduces π .
4. Schnorr = proof of knowledge of $\log_G X$. SNARK = the same, for any relation.

Say

- In Bitcoin the validator holds everything it checks: the witness sits in the transaction. Here it holds only x and a short π ; the witness w never leaves the prover.
- Plain soundness would allow “a spendable note exists” — other people’s notes exist. The extractor forces the prover to *hold* a w .
- Here the relation is the ten clauses of the Action statement.

Zero knowledge: what π hides, and what it does not

all four checks

Write

1. Whatever a verifier learns from π , a simulator produces from x alone.
2. Published: nf, rk, cv, cmx.
3. Unlinkable because of: commitment hiding, the keyed nullifier PRF, key re-randomisation — not π .
4. Zero knowledge: necessary, not sufficient. A perfect proof beside nf = cmx hides nothing.

Say

- The public inputs go on chain. That they cannot be linked back to the note and the key rests on Parts C and E, not on π .
- Bitcoin's witness is the secret's public shadow: a signature, a script preimage. Here the witness never leaves the prover; the shadow is the public-input tuple.

Write

1. Succinct: proof size logarithmic in the circuit — commitments and openings, never the polynomials. One linear MSM, shared across a batch.
challenge $:= \text{Hash}(\text{everything sent so far})$ (Fiat–Shamir)
2. Proof of knowledge + Fiat–Shamir = signature.

Say

- Halo 2's verifier still pays one linear multi-scalar multiplication, shared across a batch: a SNARK in the proof-size sense.
- A commitment is an input of the hash, so it is fixed before the challenge that tests it exists. The proof becomes one static object; verification recomputes the hashes.
- A full node that has never met Alice verifies her Action long after it was made, from the proof string alone. Schnorr is Fiat–Shamir applied to the log relation.

Arithmetisation: the claim becomes a table

all four checks

row	advice			fixed					instance	
	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate
0	3	3	9	1	0	0	-1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	-1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	-1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	-35	$s + 5 = 35$

Write

1. Toy claim: “I know x with $x^3 + x + 5 = 35$ ” over $\mathbb{F}_{97}$ ($x = 3$).
 $q_M a b + q_L a + q_R b + q_O c + q_C + \text{PI} = 0$ on every row
2. Copy constraints: $a_0 = b_0 = b_1 = b_2$, $c_0 = a_1$, $c_1 = a_2$, $c_2 = a_3$.
3. Range check = lookup into a table, not algebra.

Say

- One operation per row, one gate equation evaluated on every row.
- Advice is the prover's secret trace. Fixed columns are the circuit: the selectors pick which operation a row performs. The instance column carries the public 35.
- Copy constraints wire the rows: the x in row 0 is the x in rows 1 and 2; each row's output feeds the next.
- Bitcoin Script is executed by every validator; a circuit is never executed, only checked.

Ask

Which column does the verifier never see?

Answer

Advice: the prover's trace. Fixed columns are the circuit, known to everyone; the instance column is the public 35.

Rows become polynomials

all four checks

Write

1. Roots of unity in $\mathbb{F}_{97}$: $\omega = 22$, $\omega^2 = 96 = -1$, $\omega^4 = 1$; row i lives at $\omega^i \in H = \{1, 22, 96, 75\}$.

$$a(1) = 3, a(22) = 9, a(96) = 27, a(75) = 30 \implies a(X) = 90 + 61X + 22X^2 + 24X^3$$

$$G(X) := q_M ab + q_L a + q_R b + q_O c + q_C + \text{PI}, \quad \deg G \leq 9$$

$$\text{every gate holds} \iff G(\omega^i) = 0 \text{ for all } i \iff G \text{ vanishes on } H$$

Say

- The advice column a becomes the unique polynomial of degree below 4 through its four values: the column *is* the polynomial, sampled at the rows. Likewise b , c and the public columns.
- The value $G(\omega^i)$ is exactly row i 's gate equation.
- A Merkle root also folds many rows into one object, but it opens one leaf at a time; a polynomial can be probed anywhere.

Write

$$Z_H(X) := \prod_{h \in H} (X - h) = X^4 - 1$$

$$G \text{ vanishes on } H \iff Z_H \mid G \iff G = t \cdot Z_H$$

1. Honest: $\deg G = 9$, $\deg t = 5$, remainder 0.
2. Cheat, $c_0 = 10$: row 0 reads $3 \cdot 3 - 10 = -1 \neq 0$, so $Z_H \nmid G^*$.
$$D(X) := G^*(X) - t^*(X) Z_H(X) = 24(1 + X + X^2 + X^3) \neq 0$$

Say

- Factor theorem, applied four times: G vanishes on H if and only if Z_H divides G .
- A correct computation *is* a polynomial identity; a wrong one cannot be. Copy constraints get the same treatment (a running product equal to 1 when the wiring holds), and so do lookups.
- Bitcoin's validator checks each input's script on its own; here every row collapses into one identity that holds everywhere or fails almost everywhere.

One random point: the protocol

all four checks

Write

1. Prover commits to a, b, c and t ; verifier knows $q_M, \dots, q_C$ and PI.
2. Transcript hash draws $z \in \mathbb{F}$ (Fiat–Shamir; BLAKE2b in the deployed transcript).
3. Prover opens $a(z), b(z), c(z), t(z)$ with short proofs.
4. Verifier forms $G(z)$, computes $Z_H(z) = z^4 - 1$, checks
$$G(z) \stackrel{?}{=} t(z) Z_H(z)$$
5. Honest, $z = 20$: $Z_H(20) = 46$, $t(20) = 67$, $G(20) = 75 = 67 \cdot 46$. ✓

Say

- Every honest prover passes at every z , because $G = t Z_H$ holds as polynomials.
- Bitcoin's validator re-executes the script; this one evaluates one equation at one point.

One random point: why a lie is caught

all four checks

Write

1. Cheat: $D := G^* - t^* Z_H \neq 0$, frozen before z is drawn.

$$\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|} \quad (\text{Schwartz-Zippel})$$

2. Roots of $D = 24(1 + X + X^2 + X^3)$: $\{22, 75, 96\}$; 3/97 for that t^* , 9/97 for any admissible t^* .
3. At $z = 20$: $G^*(20) = 20$ but $t^*(20) Z_H(20) = 64$. Caught.
4. Over $\mathbb{F}_{p_{\text{Pallas}}}$, $\deg D \approx 2^{14}$: about 2^{-240} ; the discrete-log layer gives about 126 bits.

Say

- The commitments pin G^* and t^* before z is drawn: D is fixed, nonzero, with at most $\deg D$ roots.
- The 2^{-240} is one Schwartz–Zippel term, not the system's security level.
- A false statement disagrees with the truth almost everywhere; one probe lands where it disagrees.

Ask

Why 3/97 for this cheat but 9/97 in general?

Answer

This D has exactly three roots, the rows where the tampered gate still held; the general bound is $\deg D \leq 9$.

Committing to a polynomial: the problem

all four checks

Write

1. Needed: a short commitment P to f , and a short proof that $f(x) = v$ at a drawn x for the committed f .
 $P = \text{Commit}(f; r) = \langle \vec{a}, \vec{G} \rangle + [r] H \in \mathbb{G}$
2. One Vesta point for 2^{11} coefficients. Binding under discrete log; hiding by $[r]H$; no secret behind $\vec{G}$.
3. Merkle: one entry in $\log_2 d$ hashes. IPA: a combination of all entries in $2 \log_2 d$ points.

Say

- The random-point argument assumed the polynomials were pinned before z was drawn.
- Pedersen, vectorised: commit to the coefficient vector with generators $\vec{G}$ and H hashed into existence.

Committing to a polynomial: the opening claim

all four checks

Write

$$f(x) = \langle \vec{a}, \vec{b}(x) \rangle, \quad \vec{b}(x) := (1, x, x^2, \dots, x^{d-1})$$

$$P + [v]U = \langle \vec{a}, \vec{G} \rangle + [r]H + [\langle \vec{a}, \vec{b} \rangle]U$$

1. Sending $\vec{a}$: d elements. Inner-product argument: $2 \log_2 d$ points, 22 for $d = 2^{11}$.

Say

- Evaluation is an inner product. With a second hashed generator U , the opening claim becomes one group equation.
- The inner-product argument is the Bulletproofs family known from range proofs.
- A Merkle proof names one leaf; an IPA opening is one number computed from every coefficient, at a point chosen after they were fixed.

Ask

How many points does the opening cost for $d = 2^{11}$?

Answer

$$2 \log_2 d = 22.$$

The inner-product argument: one fold

all four checks

Write

1. Split: $\vec{a} = (\vec{a}_{\text{lo}}, \vec{a}_{\text{hi}})$, likewise $\vec{G}$ and $\vec{b}$.

$$L := \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + [\ell] H + [\langle \vec{a}_{\text{lo}}, \vec{b}_{\text{hi}} \rangle] U$$

$$R := \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle + [r_R] H + [\langle \vec{a}_{\text{hi}}, \vec{b}_{\text{lo}} \rangle] U$$

2. Challenge u ; fold:

$$\vec{a}' := u \vec{a}_{\text{lo}} + u^{-1} \vec{a}_{\text{hi}}, \quad \vec{G}' := u^{-1} \vec{G}_{\text{lo}} + u \vec{G}_{\text{hi}}, \quad \vec{b}' := u^{-1} \vec{b}_{\text{lo}} + u \vec{b}_{\text{hi}}$$

$$\langle \vec{a}', \vec{G}' \rangle = \langle \vec{a}, \vec{G} \rangle + u^2 \langle \vec{a}_{\text{lo}}, \vec{G}_{\text{hi}} \rangle + u^{-2} \langle \vec{a}_{\text{hi}}, \vec{G}_{\text{lo}} \rangle$$

$$P + [v] U + [u^2] L + [u^{-2}] R = \langle \vec{a}', \vec{G}' \rangle + [r + u^2 \ell + u^{-2} r_R] H + [\langle \vec{a}', \vec{b}' \rangle] U$$

Say

- Both parties fold in step: the prover folds $\vec{a}$ with weights (u, u^{-1}) , the verifier folds $\vec{G}$ and $\vec{b}$ with the inverse weights, by public arithmetic, never seeing $\vec{a}$.
- The old claim plus $[u^2]L + [u^{-2}]R$ is the same claim at half the length.
- This is the textbook (Bulletproofs) form; a Bulletproofs range proof folds its vectors by exactly this move.

The inner-product argument: the recursion

all four checks

Write

1. After $k = \log_2 d$ rounds every vector has length 1.
2. Proof: $2k$ points (L_j, R_j) , final scalar a , blinder $r^* = r + \sum_j (u_j^2 \ell_j + u_j^{-2} r_j)$.

$$P + [v] U + \sum_{j=1}^k ([u_j^2] L_j + [u_j^{-2}] R_j) \stackrel{?}{=} [a] G^{(0)} + [r^*] H + [a \cdot b^{(0)}] U$$

$$b^{(0)} = \prod_{\ell} (u_{\ell}^{-1} + u_{\ell} x^{2^{\ell-1}}) \quad O(\log d) \text{ field operations}$$

$$G^{(0)} = \langle \vec{s}, \vec{G} \rangle \quad \text{an MSM of length } d$$

Say

- The verifier accumulates the cross terms into the commitment and checks one group equation.
- The point $G^{(0)}$ is the one linear step; the node shares it across a batch. Recursion would defer it into an accumulator, which is not deployed.
- Binding: two openings of one commitment would exhibit a non-trivial discrete-log relation among $\vec{G}, H$ on Vesta.

What the fold hides, and the deployed normalisation

all four checks

Write

1. Folding alone is not zero knowledge: the final a is a challenge-dependent linear form in the coefficients.
2. Deployed: mask f by a random polynomial vanishing at the opening point, commit as S , draw two more challenges, then fold.
3. Deployed weights: $(1, u_j^{-1})$ on coefficients, $(1, u_j)$ on $\vec{b}$ and $\vec{G}$; accumulate $[u_j^{-1}] L_j + [u_j] R_j$.
4. Same $2k$ points, plus S and two final scalars.

Say

- Blinding P and the round points hides those points, not a .
- The two conventions are related by per-round rescaling; the previous board's form is the textbook one.
- A Bitcoin signature exposes its public key by design; here even the last scalar of an opening must be masked.

The fold by hand: two rounds

all four checks

Write

1. Toy: $p(X) = 5 + X + X^3$ opened at $x = 3$, so $v = 35$. Group: $\mathbb{Z}_{97}$ written additively, $[k]G = kG \bmod 97$.

$$\vec{a} = (5, 1, 0, 1), \quad \vec{b} = (1, 3, 9, 27), \quad \vec{G} = (2, 3, 5, 7)$$

$$H = 11, \quad U = 13, \quad r = 17, \quad P = \langle \vec{a}, \vec{G} \rangle + [r]H = 13$$

round	L_j	R_j	u_j	$\vec{a}'$	$\vec{b}'$	$\vec{G}'$	fold check
$j = 2$ ($\ell_2, r_2 = 19, 23$)	13	4	3	(15, 68)	(92, 82)	(48, 22)	$25 = 25$
$j = 1$ ($\ell_1, r_1 = 29, 31$)	52	58	5	(11)	(21)	(42)	$12 = 12$

2. Fold check per row: $\langle \vec{a}', \vec{G}' \rangle + [r^*]H + [\langle \vec{a}', \vec{b}' \rangle]U = P + [v]U + [u^2]L + [u^{-2}]R$.

Say

- The stand-in group $\mathbb{Z}_{97}$ has no discrete-log hardness; the numbers show only the fold's algebra. Each row's check is the one-fold identity, instantiated.

Write

1. Prover sends $a = 11$ and $r^* = 30$.

$$\vec{s} = (u_1^{-1}u_2^{-1}, u_1u_2^{-1}, u_1^{-1}u_2, u_1u_2) = (13, 34, 20, 15), \quad G^{(0)} = \langle \vec{s}, \vec{G} \rangle = 42$$

$$b^{(0)} = g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2) = 21$$

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 = 12$$

$$[a]G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U = 12$$

Say

- The verifier works from the challenges alone: $\vec{s}$, $G^{(0)}$ and $b^{(0)}$ need nothing from the prover.
- Both sides come to 12: accepted. Four coefficients were never sent; four points and two scalars were.
- On Vesta the same equations are hiding and binding, and $G^{(0)}$ is a length- 2^{11} multi-scalar multiplication.

Write

1. Native field: $\mathbb{F}_{p_{\text{Pallas}}}$, the *base* field of Pallas.
2. Native points: `cm`, `rk`, `cv`, `ak`, `pkd` — coordinates in $\mathbb{F}_{p_{\text{Pallas}}}$, group law by native arithmetic.
3. Column polynomials committed on Vesta, whose *scalar* field is exactly $\mathbb{F}_{p_{\text{Pallas}}}$.

Say

- The group law is native arithmetic, not emulated; the committed coefficients are the circuit's own field elements.
- Bitcoin's `secp256k1` is one curve doing one job: its validator verifies signatures, never circuits.

The Pasta cycle, once

all four checks

Write

1. Pallas over $\mathbb{F}_{p_{\text{Pallas}}}$ has order p_{Vesta} ; Vesta over $\mathbb{F}_{p_{\text{Vesta}}}$ has order p_{Pallas} .
2. Vesta-committed proof: Vesta points, coordinates in $\mathbb{F}_{p_{\text{Vesta}}}$ $\rightarrow$ verified natively by a circuit over $\mathbb{F}_{p_{\text{Vesta}}}$, committed on Pallas.
3. Its proof: Pallas points, coordinates in $\mathbb{F}_{p_{\text{Pallas}}}$ $\rightarrow$ verified natively by a circuit over $\mathbb{F}_{p_{\text{Pallas}}}$, committed on Vesta.
4. Not deployed: the node verifies each bundle's proof directly and batches the deferred MSMs.

Say

- Each curve's scalar field is the other's base field: that is the whole definition of the cycle.
- Proofs alternate between the two curves, each verifier a circuit, so one proof can attest to the previous one.
- Bitcoin needs no such pair; a signature is verified by arithmetic, not by another proof.

Why not Bitcoin's curve: two-adicity

all four checks

field	bits	two-adicity of $p - 1$	largest domain
$\mathbb{F}_{p_{\text{Pallas}}}$ (Pallas base)	255	32	2^{32}
$\mathbb{F}_{p_{\text{Vesta}}}$ (Vesta base)	255	32	2^{32}
secp256k1 base field	256	1	2
secp256k1 scalar field	256	6	64

Write

1. Interpolating over $n = 2^{11}$ rows needs a primitive 2^{11} -th root of unity: $2^{11} \mid p - 1$.
2. Quotient FFTs run on $8n = 2^{14}$ points at the deployed gate degree.
3. Two-adicity: the largest k with $2^k \mid p - 1$.
4. Bitcoin's prime $p = 2^{256} - 2^{32} - 977$: $p - 1$ is twice an odd number, a domain of two points.

Say

- The Pasta primes were designed for this: two-adicity exactly 32 in both, so domains of every size up to 2^{32} .
- The secp256k1 curve buys its implementation speed from $a = 0$; its prime was not chosen for evaluation domains.

Ask

Could secp256k1's base field host even the four-row toy?

Answer

No: two-adicity 1, largest domain two points.

Write

$$pp = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \dots), \quad \tau \text{ destroyed}$$

1. Whoever keeps τ forges: for any C, z, y ,
 $\pi := [(\tau - z)^{-1}] (C - [y]G)$ passes, no polynomial behind it — the toxic waste.
2. Groth16: secrets per circuit. MPC: safe if *one* participant destroyed their share. The Sapling pool still rests on that.
3. Bound (ZIP-209): each pool's chain value balance is public and may not go negative $\Rightarrow$ counterfeit capped at the pool's exit.
4. Orchard, Ironwood: IPA generators hashed from a public domain tag; nothing to leak, nothing to destroy.

Say

- A forged Sapling proof creates value only inside the Sapling pool and can withdraw at most what that pool publicly holds — the turnstile, Part H.
- Bitcoin never had a setup; Sapling has one; Orchard returns to none.

Three generations

all four checks

	Sprout	Sapling	Orchard, Ironwood
SNARK	BCTV14 (original)	Groth16	Halo 2
curve	BN254	BLS12-381, Jubjub in-circuit	Pallas, Vesta
setup	six-party MPC	large MPC	none
hash in circuit	SHA-256	Pedersen	Sinsemilla
proof	296 B	192 B	$2720 + 2272a$ B
verification	constant	three pairings	$O(\log n) +$ shared MSM

Write

1. Sprout: BCTV14 soundness flaw found after deployment; re-proved with Groth16, deprecated.
2. Sapling: Groth16 over BLS12-381, Jubjub inside the circuit.
3. Orchard: proof grows because transparency costs $2 \log_2 n$ points.

Say

- Sapling: a far faster prover, a per-circuit ceremony, no curve cycle.
- Bitcoin changed its signature scheme once, by opt-in; Zcash has carried shielded payments through three proof systems, each with its own pool — the third with two, Orchard and Ironwood, on one circuit.

The Orchard circuit by the numbers

all four checks

rows	$2^{11} = 2048$; one fixed circuit, every Action an instance
columns	ten advice, fixed columns, a Sinsemilla lookup table
proof per bundle	$2720 + 2272 a$ B: 4992 for one Action, 7264 for Alice's two
of which IPA	22 pair points and one blinding commitment, 32 B each
proving	a few seconds on commodity hardware
verifying	$O(\log n)$ per proof, plus one MSM shared across a batch

Write

1. One aggregate proof per bundle: per-Action term = that Action's column commitments and evaluations; constant term = the shared quotient, multipoint and IPA data.
2. Spend-auth signature: 64 B. One-Action proof: 78 signatures' worth.

Say

- Bitcoin's witness cost is per input and grows with the script it satisfies; here it is per bundle, logarithmic in the circuit, and independent of the anonymity set.

Five gadget families, three circuit versions

all four checks

Write

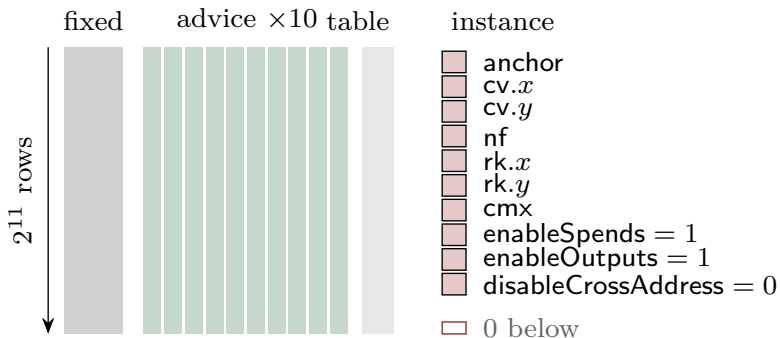
1. ECC: $rk = ak + [\alpha] G^{\text{SpendAuth}}$ (A6), $pk_d = [ivk] g_d$ (A7), cv (A4), the nullifier's point (A5).
2. Sinsemilla: generator table as a lookup; per 10-bit chunk one lookup and two incomplete additions; **NoteCommit** (A1, A2), **Commit**^{ivk} (A7).
3. Merkle path: 32 layer-tagged hashes, a conditional swap per layer by one bit of the position, $root = anchor$ gated by v_{old} (A3).
4. Poseidon: the nullifier PRF; only non-linearity $x \mapsto x^5$ (A5).
5. Decomposition: $v \in [0, 2^{64})$ and the sign bit (A4); canonicity of witnessed encodings.
6. Three versions: original, a correction, the Ironwood circuit with the cross-address input. Verifying key selected by branch.

Say

- Canonicity of witnessed encodings is distinct from the node's parsing of public encodings, Part E.
- Consensus selects the verifying key by branch, so every Action in either pool now verifies against the Ironwood key.
- Bitcoin's validator runs whatever script an output declares; this one runs a verifier keyed to one fixed circuit.

Alice's Action as an instance

all four checks



Write

1. Instance (Action 1: 2 ZEC $\rightarrow$ Bob 1.5): eight public inputs, ten field elements; points by coordinates; flags as the Ironwood component sets them.
2. Advice: both notes; 32 siblings and the position; $ak, nk, r_{ivk}, ivk; \alpha; rcv;$ (magnitude, sign) = (50 000 000, +1).
3. Copy constraints tie each instance cell to the advice cell holding the same value.

Say

- The verifier computes the instance commitment itself, so a proof for other values fails the opening.
- Bitcoin's witness — scriptSig or witness field — is in the clear; here the witness fills advice cells nobody reads.

Ask

Checkpoint: what could a holder of the Sapling ceremony's toxic waste do, what stops the analogue in Orchard, and what bounds the damage?

Answer

- Forge accepting Sapling proofs of false statements: counterfeit inside the Sapling pool, invisible to every verifier. Not decrypt notes — soundness and zero knowledge are separate guarantees.
- In Orchard there is no trapdoor to hold: the generators are hashed, and binding reduces to discrete log.
- Under ZIP-209 each pool's chain value balance is public, and a block that would drive it negative is invalid — counterfeit is capped at the pool's exit.

Part F — Delivery and light clients

The note plaintext: what travels to Bob

exists

Write

1. $np := (0x03, d, v, rseed, memo)$
2. $1 + 11 + 8 + 32 + 512 = 564$ bytes
3. $rseed \rightarrow esk \quad (PRFexpand_{rseed}(0x04 \parallel \rho))$
4. $rseed \rightarrow \psi \quad (PRFexpand_{rseed}(0x09 \parallel \rho))$
5. $rseed \rightarrow rcm \quad (\text{next board})$
6. $\rho := nf_{old}$

Say

- The chain stores only cmx ; Alice delivers the note plaintext to Bob herself.
- Lead byte $0x03$ is the Ironwood note, the only value the Ironwood pool admits.
- The 32-byte seed is the note's whole randomness; $\rho := nf_{old}$ binds it to its Action. The memo never appears in the clear.
- Bitcoin: amount and script are public, and so is an `OP_RETURN` note.

The Ironwood trapdoor: one hash over every note field

exists

Write

1. Ironwood pool (0x03): $\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(\text{pre}_{\text{rcm}}))$
2. $\text{pre}_{\text{rcm}} := 0\text{x0B} \parallel \text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{256}(\rho) \parallel \text{LE}_{256}(\psi)$
3. $1 + 32 + 32 + 8 + 32 + 32 = 137$ bytes
4. Orchard pool (0x02): $\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0\text{x05} \parallel \text{LE}_{256}(\rho)))$
5. $1 + 32 = 33$ bytes

Say

- The one note-level difference between the pools is how rcm leaves the seed; esk and ψ are byte-identical, and the lead byte says which derivation applies.
- Every note field enters the Ironwood preimage, so rcm is derived last, after ψ .
- The commitment is unchanged: same **NoteCommit**, same tree construction, each pool its own instance (ZIP-229).
- Bitcoin: an output has no trapdoor; there is nothing hidden to bind.

Write

1. Sinsemilla binding = discrete log $\rightarrow$ falls to a quantum computer
2. PRFexpand as a random oracle; every field enters pre_{rcm}
3. derivation + NoteCommit = post-quantum binding
4. *provided a verifier checks the derivation* — no deployed clause does
5. Ironwood note: recoverable; Orchard-/Sapling-pool note: not

Say

- Sinsemilla collision resistance binds NoteCommit; a large quantum computer breaks it (Part I), and then NoteCommit alone binds nothing.
- Any change to a committed note re-randomises rcm; two honest note-trapdoor pairs on one commitment is an oracle collision, with no group structure for Shor.
- The claim is conditional on a future verifier check of the derivation. This is what makes an Ironwood note recoverable (Parts H and I).
- Bitcoin: nothing in an output is hidden, so nothing must stay bound after a break.

Sealing the note: one-shot Diffie–Hellman, then an AEAD

exists

Write

1. Bob's address (d, pk_d) , $\text{g}_d = \text{GroupHash}(d)$, $\text{pk}_d = [\text{ivk}] \text{g}_d$
2. $\text{epk} := [\text{esk}] \text{g}_d$
3. $s := [\text{esk}] \text{pk}_d$
4. $K_{\text{enc}} := \text{KDF}(s, \text{epk})$
5. $C_{\text{enc}} := \text{Enc}_{K_{\text{enc}}}(\text{np})$ 564 + 16-byte tag = 580 bytes
6. $[\text{ivk}] \text{epk} = [\text{ivk}][\text{esk}] \text{g}_d = [\text{esk}] \text{pk}_d = s$

Say

- KDF: BLAKE2b-256 over the encodings of s and epk . AEAD: ChaCha20-Poly1305, fixed all-zero nonce, empty associated data — a fresh key for every note.
- Bob (with ivk) and Alice (with esk) both reach s ; either recovers K_{enc} from the published $(\text{epk}, C_{\text{enc}})$.
- Bitcoin: BIP-352 runs the same ECDH (input keys as esk , no epk) to derive an output key shown in the clear; here the shared secret encrypts the note, memo included.

The second ciphertext: the sender's own copy

exists

Write

1. $\text{ock} := \text{BLAKE2b-256}(\text{ovk} \parallel \text{cv} \parallel \text{cmx} \parallel \text{epk})$
2. $C_{\text{out}} := \text{Enc}_{\text{ock}}(\text{pk}_d, \text{esk}) \quad 64 + 16 = 80 \text{ bytes}$
3. ovk holder: $s = [\text{esk}] \text{pk}_d \rightarrow$ reads the note as Bob does
4. $\text{ovk} = \perp \rightarrow$ random bytes under a random key
5. Action publishes $(\text{epk}, C_{\text{enc}}, C_{\text{out}})$ beside $\text{cv}, \text{nf}, \text{rk}, \text{cmx}$

Say

- Eighty bytes buy a sent history: from $(\text{pk}_d, \text{esk})$ the holder of ovk re-derives s and reads the note as Bob does.
- Binding $\text{cv}, \text{cmx}, \text{epk}$ into ock ties C_{out} to its own Action; with $\text{ovk} = \perp$ no outgoing key ever opens it.
- The node never opens any of the three; no clause of the Action statement mentions them.
- Bitcoin: the chain is the sender's copy; here a sent history needs a kept key.

Finding your money: detection is successful decryption

exists

Write

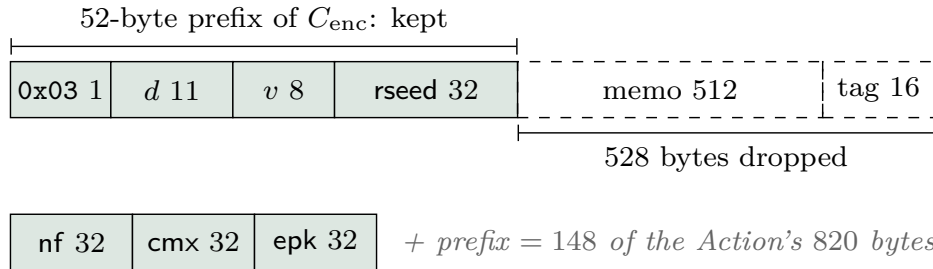
1. for *every* Orchard-protocol output:
2. $s := [\text{ivk}] \text{epk}$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$, $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$
3. tag rejects a wrong key $\rightarrow$ an open is a *candidate*
4. check 1: esk from rseed , $[\text{esk}] g_d = \text{epk}$
5. check 2: note from (d, v, rseed) and $\rho := \text{nf}_{\text{old}}$, $\text{Extract}(\text{NoteCommit}_{\text{rcm}}(\dots)) = \text{cmx}$
6. cost: one key agreement per output per ivk + every output's bandwidth

Say

- Nothing on chain names a recipient, so there is nothing to ask for: Bob's wallet trial-decrypts every Orchard-protocol output with ivk .
- An open is a candidate, not yet a note; the two re-derivations close the obvious forgeries.
- The cost is a count, not a clock. One ivk scans both pools.
- Bitcoin: a lookup over public scripts; here, a decryption attempt on every output.

Compact blocks (ZIP-307): what a light client downloads

exists, not twice



Write

1. compact Action = nf || cmx || epk || $C_{\text{enc}}[0..52)$
2. $3 \cdot 32 + 52 = 148$ of 820 bytes
3. dropped: memo, tag, C_{out} , cv, rk, anchor, proof, signatures

4. per block, per pool: end-of-block tree size

Say

- A compact block keeps only what three tasks need: detect a payment, detect a spend of one's own notes, update witnesses.
- Bitcoin: an SPV client downloads headers and asks for what matches; this client downloads every compact block in range and asks for no particular output.

Accepting a note without its tag

exists

Write

1. $np := \text{ciphertext}[0..52) \oplus \text{ChaCha20}_{K_{\text{enc}}}(\text{nonce} = 0)[64..116)$
2. bare keystream, past block 0 (the tag key's block)
3. accept iff: 1. the 52 bytes parse as a compact note plaintext; 2. recomputed commitment = served cmx; 3. esk from rseed reproduces epk
4. commitment binding stands in for the tag

Say

- Truncation at 52 bytes discards the Poly1305 tag, so the wallet applies the bare keystream past block 0, which the full AEAD reserves for the tag key.
- Bob holds neither cv nor the binding signature; under commitment binding, no sender can make him accept a note the chain did not commit to.
- Bitcoin: BIP-37 hands each peer a Bloom filter of the wallet's scripts; BIP-157 matches a per-block filter locally, then fetches the matching blocks. Here the “filter” is ivk, applied to every block, and no match is ever requested.

Write

1. scan stage: **less** — every compact block in range fetched; server sees range bounds + timing, never a per-block match
2. after detection: **more** — full-transaction fetch names an exact txid (BIP-157: a whole block)
3. ZIP-307 on that fetch: decoys (SHOULD); every transaction of any block touched (SHOULD); tell the user (MUST)
4. deployed libraries: no decoy policy

Say

- A BIP-157 client reveals each block it fetches after a filter hit; here the server never sees a per-block match.
- The memo and C_{out} live in the full transaction, which compact blocks omit.
- Structural: asking for ranges reveals where the client stopped; fetching what it detected reveals what it detected.

Placing the note: a position from public data

exists, not twice

Write

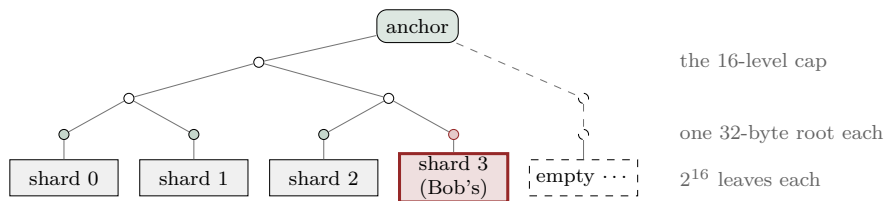
1. $\text{pos}(\text{output } i \text{ of } tx) = \text{start-of-block size} + \text{offset}(tx) + i$
2. $\text{start size} = \text{stated end-of-block size} - \text{the block's own outputs}$
3. $\text{offset}(tx) = \text{outputs of the block's earlier transactions}$
4. append *every* cmx; mark own
5. spend: every compact nf vs own unspent nullifiers, locally

Say

- Trial decryption tells Bob that an output is his; the tree needs to know where. The position is arithmetic on public data, never a replay.
- The stated size is load-bearing: a note without a position has no witness.
- Bitcoin: an SPV client's branch arrives because its filter (BIP-37), its block fetch (BIP-157), or — on Electrum — the txid itself named its interest. Bob names nothing.

Shards and the cap: a witness without the whole tree

exists



Write

1. depth 32, cut at 16: shards of 2^{16} leaves, at most 2^{16} of them, 16-level cap of shard roots
2. own shard + unfinished tip shard: hashed from leaves
3. every completed shard: one root from the server
4. path: low 16 siblings from leaves, high 16 from roots
5. unbegun shard = per-level constant

Say

- Bob hashes his own shard from its leaves and takes every completed shard as one root from the server.
- Bitcoin: an SPV client is handed one branch by its peer. Bob folds his own from public roots and asks for nothing — asking for one note's path would say which note is his. Only the anchor is ever public.

Checkpoints: the tree as of a block

exists

Write

1. checkpoint at height h = last-leaf position as of block h + marks dropped since the previous one
2. uses: fold a path against block h 's root; truncate to it on a reorganisation
3. *shared* = present in every pool's tree at that height
4. bounded recent set + durable ones exempt from pruning
5. a block appending nothing need not create one

Say

- A checkpoint is the wallet's record of its tree at a block boundary, identified by the block's height.
- When one tree gains a checkpoint at a height, the others gain one too, so a single height names a state of every tree.
- Bitcoin: Bitcoin Core's checkpoints are hard-coded block hashes inside the node; these are wallet-local tree snapshots that consensus never sees.

Write

1. frontier of a tree of size n = rightmost path: last leaf + one sibling hash per filled subtree up to the root
2. wallet birth: frontier of the block *before* the birthday
3. batch seed: chain state of the block before it, checked against the first block's own end-of-block size
4. contradiction $\rightarrow$ batch mis-sequenced; size contradicted by the block's contents = continuity error $\rightarrow$ reorganisation path
5. note tree-ready: own shard's block extent + tip range scanned

Say

- The frontier seeds a shard tree, so no block below the birthday is ever replayed.
- Bitcoin: a restored wallet also rescans from its birthday but needs no tree state: the UTXO set is public; a peer answers a filter, or an Electrum server by address.

What the server learns: the schedule

none — privacy of the scan

observable	reveals	mitigation status
peer IP, connection timing	endpoint, activity windows	Tor capability; deployment unknown
lowest range start	approximate account birthday	none
range boundaries	scan state and progress	bucketing proposed, unimplemented

Write

1. keys never travel; requests do
2. ZIP-307's goal: payment-detection privacy against an honest-but-curious proxy; spending out of scope

Say

- The schedule is what leaks; the content holds.
- Bitcoin: BIP-157 shows a peer the same schedule, then every block it fetches.

What the server learns: the content channels

none — privacy of the scan

observable	reveals	mitigation status
fetch after detection	transactions of interest	decoys a SHOULD; no library policy
submission via the server	the whole transaction, linked to origin	circuit isolation; deployment unknown
transparent-address calls	addresses outright	jitter (one-use addresses only)

Write

1. detection stays on the device
2. three requests carry content, each sent only after the wallet decided to

Say

- Bitcoin: BIP-37 leaks addresses up to the false-positive rate, BIP-157 its matched blocks; here content leaks only through what the wallet sends after detecting.

Write

1. no request carries key material: heights, hashes, txids, transparent addresses, raw transactions only
2. Tor client with circuit isolation: in the wallet library; default routing not determinable
3. jitter: exponential intervals, shuffled order — ephemeral (ZIP-320) address checks only; none for fetch-on-detect
4. bucketing: range start rounded down to a multiple of N — ZIP-307's proposal, unimplemented

Say

- Trial decryption and nullifier matching run on the device.
- Circuit isolation: submission need not share a circuit with scanning.
- None of this is a property of compact-block contents, and none of it is inevitable: Tor, bucketing, jitter and cover traffic can each change it.

Write

1. witness: 32 siblings, folded on the device from own shard's leaves + other shards' roots, against a checkpointed block
2. the path = private input of the membership clause
3. anchor: any main-chain root since the pool's activation, no window
4. proving decoupled from broadcasting; appending alters no old root
5. older anchor: no soundness hole; a second spend is the nullifier's problem

Say

- The witness is folded on the device; the anchor is Part C's.
- Bitcoin: an input names one txid and index, and the validator looks it up. The anchor names a tree; the path into it never leaves the device.

Which anchor: the wallet's policy, and whose reasons

exists

Write

1. $H = \text{target height} = \text{tip} + 1$
2. anchor = newest shared checkpoint at or below $H - 3$ (wallet policy, not consensus)
3. spendable: 3 confirmations trusted, 10 untrusted, from the mined block; shard fully scanned; note's block at or below the anchor

Say

- Consensus admits any main-chain anchor since the pool's activation; $H - 3$ is ZIP-315 wallet policy. ZIP-315's stated reasons and the Guide's anonymity-set argument are Part C's.
- Bitcoin: a wallet also waits for confirmations, but which block it "anchors" to is nobody's business, because the input is named either way.

Write

1. expiry height (ZIP-203): 0 = none; included above it → block invalid
2. wallet default: target +40
3. store → lock inputs → broadcast, one call
4. expired unmined → inputs released, notes spendable again

Say

- A uniform delta, like a uniform fee, is not a fingerprint.
- The wallet persists the transaction and locks its inputs before the network sees it.
- Bitcoin: a stuck transaction has no consensus expiry — mempools drop it by policy after a timeout, or RBF or CPFP bumps it. Here it expires by rule, and its inputs come free.

Write

1. reorganisation past the anchoring block → anchor invalid
2. → truncate to a checkpoint, rescan, rebuild
3. rebuilt transaction re-reveals the *same* nullifiers (deterministic derivation)
4. re-anchoring, txid unchanged: specified (ZIP-374 v2, anchors in authorizing data); wallet support pending

Say

- The notes are the same, so the serial numbers are.
- Bitcoin: a reorganised transaction is simply mined again, since it names its inputs and no anchor. Here a struck anchor means a rebuild.

Three openings on Alice's transaction

exists

ciphertext	opened by	with	yields
Action 1, C_{enc}	Bob	ivk	$(d, 1.5 \text{ ZEC}, \text{rseed})$; the memo, once fetched
Action 2, C_{enc}	Alice	ivk^{int}	change 0.4999 ZEC, at her internal address
Action 1, C_{out}	Alice	ovk	$(\text{pk}_d, \text{esk})$, hence Bob's note as he reads it
Action 2, C_{out}	nobody	(no key)	random: the wallet default seals change under no ovk

Write

1. Bob: cmx and epk check, position recorded; with nk a nullifier absent from the set; from a stranger: untrusted, 10 confirmations
2. Alice: change by trial decryption under ivk^{int} , never through ovk ; internal notes trusted: 3 confirmations
3. Alice: $C_{\text{out}} \rightarrow$ what she sent Bob (sent history; an auditor's, given ovk)

Say

- Bitcoin: Bob's payment and Alice's change are the two outputs everyone reads, and a memo would be an OP_RETURN.

Ask

Checkpoint: Bob's light client holds `ivk` and the 52-byte prefixes: what can the server still infer, and what would a Bitcoin SPV client have given away instead?

Answer

- Still inferred: the endpoint and when it is active; the earliest range start, about his birthday; the range bounds, his progress; the exact txid, if he fetches the full transaction for the memo; the whole transaction before broadcast, if he submits through the same server.
- Never: which outputs opened, their values, or ivk itself.
- A BIP-37 client hands each peer a Bloom filter of its scripts: addresses, balance, and graph, up to the false-positive rate. A BIP-157 client narrows that to the blocks it fetches after a match — still by asking.

Part H — The transaction and the turnstile

A v6 transaction beside a Bitcoin transaction

all four

	Bitcoin	Zcash v6
header	version, lock time	+ version group, branch id, expiry height: 20 B in all
inputs	outpoint, scriptSig, sequence	the same bytes; ECDSA over secp256k1, no Taproot; the scriptSig outside the txid
outputs	value, scriptPubKey	the same bytes
Sapling bundle	—	spends, outputs, Groth16 proofs
Orchard bundle	—	Actions of the sealed pool
Ironwood component	—	a Actions at 820 B; flags, $v_{\text{balance}}^{\text{Ironwood}}$, anchor; one proof of $2720 + 2272a$ B; $a + 1$ signatures at 64 B

Write

- Proof: $2720 + 2272a$ B; for $a = 2$: 7264 B
- Alice: $a = 2$ Actions, 7264 proof bytes, 9166 bytes in all
- “One ledger, one transaction type, one asset”

Say

The shielded parts are components of the one transaction, not a second ledger. Alice's payment fills the Ironwood component alone; the transparent, Sapling and Orchard parts are empty.

Write

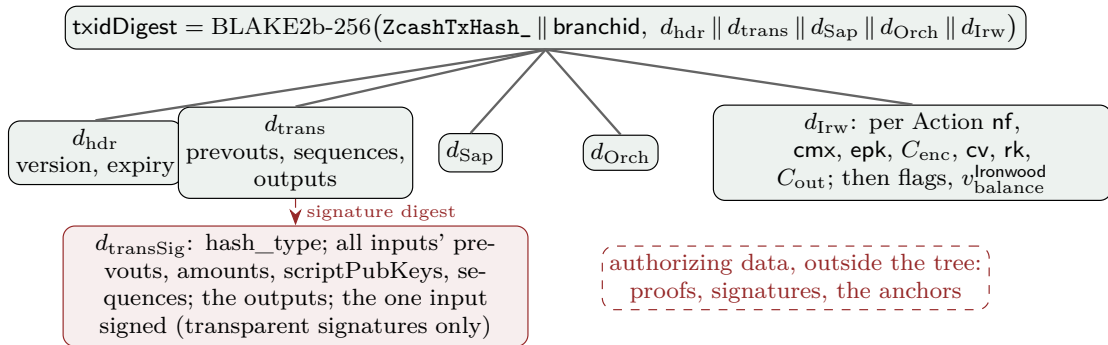
- Effecting data = everything under the txid: header; transparent outpoints, sequences, outputs; every Action's public fields; flags; balancing fields
- Authorizing data = what attests to it: proofs, signatures; from v6 the three anchors; each scriptSig
- Bitcoin: txid = SHA-256d(bytes); SegWit moves the witness out, re-commits it through the coinbase
- v6: txid = BLAKE2b-256 tree over effecting data;
hashAuthDataRoot → hashBlockCommitments (ZIP-244)
- Re-anchoring, txid unchanged: specified (ZIP-374 v2), wallet support pending
- v4, v5, v6 all valid; the seal binds whatever the version

Say

Zcash applies SegWit's move to everything that attests rather than effects: recomputing a proof or a signature changes nothing the identifier names. Because the anchor is authorizing data, re-anchoring leaves the txid unchanged.

The sighash (ZIP-244): a tree of digests

authorised



Write

- Every node = one BLAKE2b-256 under its own personalisation; the root's carries the branch id
- Signature digest = the txid tree with $d_{\text{trans}} \rightarrow d_{\text{transSig}}$
- No transparent inputs $\Rightarrow$ signature digest = txid

Say

The branch id in the root's personalisation gives the same bytes a different identity on a parallel chain. Proofs, signatures and the anchors sit outside the tree, committed through hashAuthDataRoot instead.

Write

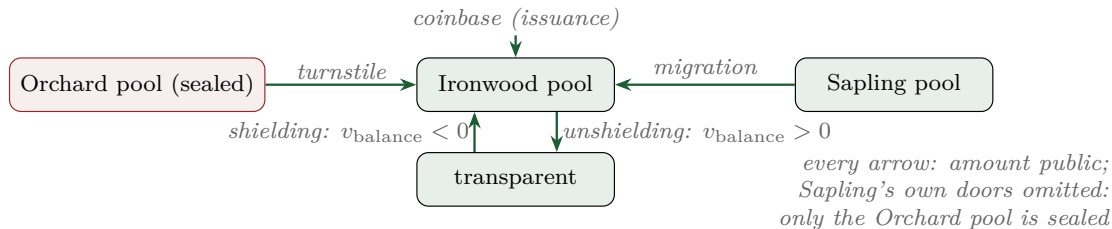
- Per Action: spend-authorisation signature under `rk`; per bundle: binding signature under `bvk` — both sign the signature digest: every Action's `nf`, `cmx`, `cv`, `rk`, ciphertexts; the flags; v_{balance}
- Bitcoin `hash_type`: `ALL`, `NONE`, `SINGLE`, each with or without `ANYONECANPAY`: six; a transparent Zcash input: exactly those six
- Taproot's `SIGHASH_DEFAULT` (a seventh spelling of `ALL`): not in ZIP-244
- Shielded: `SIGHASH_ALL`, always; no shielded `ANYONECANPAY`
- Anchor: not under the signature from `v6`; the proof carries it as a public input (A3)
- Alice: no transparent inputs $\Rightarrow$ 2 spend-auth + 1 binding signature all sign her `txid`

Say

In Bitcoin each signer picks a `hash_type`; a shielded signature has one. No Action can be transplanted into another transaction, no field altered. The signature attests to the spend, the proof pins the tree.

Pools and the doors

conserved



Write

- Shielding: transparent inputs fund $v_{\text{balance}}^{\text{Ironwood}} < 0$; the Actions spend dummies, create real notes. Alice shields 10 ZEC: $v_{\text{balance}}^{\text{Ironwood}} = -9.99985$ ZEC printed; her new note not
- Coinbase: issuance enters a shielded pool directly; every shielded coinbase output decrypts under the all-zero ovk
- Migration: Sapling $\rightarrow$ Ironwood; Orchard $\rightarrow$ Ironwood through the turnstile; both balancing fields cleartext

Say

Bitcoin has one pool and needs no doors. Zcash has several, and value crosses between them only through doors that print the amount. Wallets shield automatically, to an internal address.

Write

- $\text{balance}_P := -\sum_{\text{tx}} v_{\text{balance}}^P(\text{tx}) \geq 0$ in every block, for $P \in \{\text{Sap}, \text{Orch}, \text{lrw}\}$
- Sprout: $\sum \text{vpub_old} - \sum \text{vpub_new}$; transparent: the UTXO sum; lockbox: its accrued subsidy
- A block driving any balance negative, or the total past MAX_MONEY: invalid
- Inside a pool: counterfeit undetectable. At the exit: capped at what the pool publicly holds
- Example: pool holds only Alice's 9.99985 ZEC; Bob's +9.99985 ZEC exit empties it exactly; one zatoshi more $\Rightarrow$ block invalid

Say

Bitcoin audits by summing every coin; inside a shielded pool nothing sums the notes, and Sprout's original proof system had exactly such a flaw (Part G). Zcash audits each pool at its door: a forger withdraws at most what the pool publicly holds. The Sapling pool still rests on its Groth16 ceremony; the turnstile bounds it.

Write

- Three rules: no issuance; no circulation (`enableCrossAddress = 0`); no entry ($v_{\text{balance}}^{\text{Orchard}} \geq 0$) — whatever the transaction version
- Legacy note, lead byte 0x02: not quantum-recoverable. Ironwood note, lead byte 0x03, hashed rcm: recoverable
- Orchard pool balance can only fall; every exit metered in cleartext by $v_{\text{balance}}^{\text{Orchard}}$
- Bitcoin never retires an output type; a P2PK coin spends forever

Say

The seal is the three rules of Parts C and E; the reason is Part F's one note-level difference. Consensus stops the old notes circulating and funnels their value out through the turnstile, while owners can still spend what they hold. The turnstile is the seal's accounting shape.

Fees (ZIP-317): a function of the public shape

conserved

Write

$$fee = 5000 \cdot \max(2, logical_actions) \quad \text{zatoshi}$$

$$logical_actions = \max\left(\left\lceil \frac{\text{in bytes}}{150} \right\rceil, \left\lceil \frac{\text{out bytes}}{34} \right\rceil\right) + 2 \, nJoinSplit_{\text{Sprout}} \\ + \max(spends, outputs)_{\text{Sap}} + nActions_{\text{Orch}} + nActions_{\text{Irw}}$$

- Divisors: standard P2PKH input 150 B, output 34 B
- Alice: two Actions, $5000 \cdot \max(2, 2) = 10\,000$ zatoshi
- One Action padded to two: 10 000, the same
- Shielding with one P2PKH input: $1 + 2 = 3$ actions, 15 000

Say

Bitcoin reads the fee by subtraction; here the amounts are hidden, so the fee is a function of the public shape. An Action spends one note and creates one, so it is counted once.

Fees (ZIP-317): why a convention, and who enforces it

conserved

Write

- Computable from public fields alone
- Uniform across wallets: not a fingerprint; padding to two is free (grace window = 2)
- A convention, not consensus: relay at least the fee; template by fee ratio, capped at 4×; at most 50 unpaid actions per block
- Bitcoin: a market — bid per virtual byte and coin selection vary; both fingerprints

Say

Bitcoin's fee market is read as a fingerprint; Zcash flattens the market on purpose. A fee that depended on which notes were spent or what change came back would print that data in the amount paid. Overpaying beyond 4× buys no further priority.

The public face of Alice's transaction

all four

public field	Alice pays Bob 1.5 ZEC from a 2 ZEC note
version	v6; transparent, Sapling and Orchard parts empty
Ironwood Actions	2 (one real spend, one dummy; shuffled)
nullifiers,	2 and 2
commitments	
$v_{\text{balance}}^{\text{Ironwood}}$	+10 000 zatoshi = the fee
anchor	the Ironwood root at the newest shared checkpoint at or below $H - 3$
expiry	target height + 40 blocks, about 50 minutes
signatures	2 spend-authorisation + 1 binding
size	9166 bytes, 7264 of them proof

Write

- Bitcoin prints: input coin, both amounts, both scripts, fee by subtraction
- Here printed: the fee alone, as $v_{\text{balance}}^{\text{Ironwood}}$
- Inside the commitments: 2 ZEC, 1.5 ZEC, 0.4999 ZEC change, both addresses

Write

- Pool: transparent leg prints its amount as v_{balance} ; fully shielded prints the fee and nothing else
- Address type: unified address = bundle of receivers; wallet MUST use the most preferred it supports: Orchard-protocol (Ironwood pool) > Sapling > transparent. TEX address $\Rightarrow$ two transactions via an ephemeral transparent output, both public
- Timing: expiry = target + 40 blocks, wallets SHOULD keep it. Migration: denominations $m \cdot 10^k$ ZEC, $m \in \{1, 2, 5\}$; anchors and expiries from shared grids
- “Uniformity is the strategy”

Say

Bitcoin has the address-type and timing sliders but no pool slider: one plaintext ledger. An idiosyncratic expiry delta fingerprints a transaction as an idiosyncratic fee would. The defaults protect only while everyone keeps them.

Write

- PSBT (BIP-174 + BIP-370): Creator, Constructor, Updater, Signer, Combiner, Input Finalizer, Transaction Extractor
- PSBT cannot carry shielded: no shielded spends/outputs; no proof distinct from a signature; tolerates unknown fields
- PCZT = PSBT roles + IO Finalizer, Redactor, Prover; ten roles
- “Proving is delegated, signing is not”

Say

Hardware wallets and multisig live on PSBT. The IO Finalizer declares the inputs and outputs complete and derives the binding-signature key; the Redactor strips what later parties need not see. The Prover has no PSBT counterpart: a hardware signer cannot compute a Halo 2 proof, and a proof needs no spending key. Signers can be fully offline.

Signer versus Prover

authorised

	Signer	Prover
holds	ask, spend authority	fvk, no spending key
reads from the PCZT	α , rk; the spend's public fields and, unless redacted, its note fields	the note ($\mathbf{g}_d$, $\mathbf{pk}_d$, v , ρ , rseed), the witness, α , rcv, the anchor
computes	the sighash from the effecting data; $\mathbf{rsk} = \mathbf{ask} + \alpha$, checks it matches rk, signs	one Halo 2 proof over all Actions; anchor and flags as public inputs
verifies first	nf, cv, cmx against the note fields: exactly what it authorises	fvk owns each note; $\rho_{\text{new}} = \mathbf{nf}_{\text{old}}$
learns	what it signs	the notes' private data, not authority

Write

- $rsk = ask + \alpha$; check against rk before signing
- $\rho_{\text{new}} = nf_{\text{old}}$
- Either role runs without the other; the Prover never sees ask ; the Signer may see the note data, or have it redacted

Say

In Bitcoin there is no proof distinct from the signature, so the signer is the only role that attests. Here proof and signature are separate artefacts with separate secrets.

Write

- From v6: signatures commit to no anchor $\Rightarrow$ sign today, anchor, witness and prove at broadcast
- Delegating the Prover reveals note data, never spend authority; a Redactor strips what a later party need not see
- Bins: ZIP Draft; format released as a library; deployed wallet pipeline drives the roles end to end (Ironwood sent-output records not yet stored: backend gap)
- + FROST (Part I): “no script, so no multisig and no hardware wallets” — closed

Ask

Alice shields 10 ZEC; the next day Bob unshields 9.9997 ZEC. What can a chain analyst conclude, and what can they not?

Answer

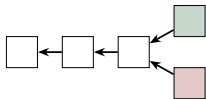
- **Conclude.** Two cleartext balancing fields, exact to the zatoshi: -9.99985 ZEC entered, $+9.99985$ ZEC left — 10 ZEC less two conventional fees of 15 000 zatoshi; the heights; the transparent addresses on each side; that the amounts match exactly and the times are close.
- **Not conclude.** That Bob's coins were Alice's. Her commitment hides her note; his nullifier is keyed by his nk ; his candidate set is every leaf under his anchor. Her note may still be resting.

Bitcoin would have handed over the path; Zcash hands over a correlation. Canonical denominations and shared timing grids exist because that correlation is evidence.

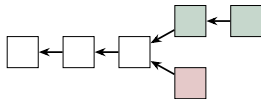
Part K — Consensus: why one fork wins, and what the chain must remember

Bitcoin's chain rule, weighed in work

not twice



(a) equal work: nodes disagree



(b) one more block: the upper branch is heavier

Write

- work of a block = $\lfloor 2^{256} / (\text{ToTarget}(\text{nBits}) + 1) \rfloor$
- best valid chain = greatest total work; ties $\rightarrow$ the block received first
- “longest” = “most work” only at constant difficulty
- nothing marks a chain as final

Say

Every block names its parent, so the node holds a tree of blocks; each branch is one consistent history, and sibling branches may conflict. A heavier branch wins however deep the fork.

Two double-spends; consensus answers one

not twice

Write

- same chain: two spends of one note $\Rightarrow$ the same nf twice $\Rightarrow$ invalid by rule; no race, no probability
- chain level: pay on the chain everyone sees, then replace that chain; no proof rules it out, only the proof-of-work race bounds it
- Bitcoin: the same two attacks; a UTXO lookup there, a nullifier lookup here

Say

Bitcoin stops a double spend in two places: a UTXO lookup on the chain it holds, and the work rule between chains. Both attacks exist here; the first changes shape and the second does not. The second is what this part measures.

Write

1. broadcast the payment to the merchant
2. mine a secret branch from the last pre-payment block, carrying a conflicting payment to yourself
3. wait until the merchant ships at n confirmations
4. extend the secret branch until it carries more work, then release it

step 4 breaks no validity rule — does the secret branch ever get ahead?

Say

Bitcoin's whitepaper analyses this attack in its §11, approximately; Rosenfeld's *Analysis of hashrate-based double-spending* is the exact counterpart, and this part follows it throughout. The released branch is a well-formed chain, merely a different one; the only question is probabilistic, and from here on its odds are computed exactly.

Write

- $\Pr[X > t] = e^{-\lambda t}$; $\Pr[X > s + t \mid X > s] = \Pr[X > t]$
- two clocks: $\Pr[X < Y] = \lambda/(\lambda + \mu)$; the winner is independent of the winning time
- honest rate p/T_0 , attacker rate q/T_0 , $p + q = 1$; both restart at every block
- next block is the attacker's with probability q , independently of everything before
- block attribution = an i.i.d. p/q coin sequence; T_0 drops out of every answer

Say

This is Rosenfeld's version of the whitepaper's race between an honest and an attacking hashrate, with its two assumptions explicit: constant hashrates and constant difficulty. An exponential clock of rate λ is memoryless: having waited s says nothing about the wait still to come. Here T_0 is the network's mean block time. How well Equihash fits the model is a deployment question (later in this part).

The negative binomial law

not twice

Write

- m = attacker blocks found strictly before the honest network's n -th block
- $P(m) = \binom{m+n-1}{m} p^n q^m$, $m = 0, 1, 2, \dots$
- first-to- n race: H_n or A_n ; no tie; $\Pr[H_n] + \Pr[A_n] = 1$
- $q = 1/5$, $n = 6$: $P(0) = 0.262144$, $P(1) = 0.314573$, $P(2) = 0.220201$, ..., $P(5) = 0.021139$; $\Pr[m \geq 6] = 0.011654$

Say

Bitcoin's whitepaper approximates the attacker's progress by a Poisson; this is the exact law. The $(m+n)$ -th trial is the n -th honest success, and the m attacker blocks sit among the first $m+n-1$ trials in any order. Counts advance one at a time, so the first-to- n race has no tie. The running numbers return in the worked race.

The catch-up theorem (Rosenfeld §3)

not twice

Write

- z = the honest lead in blocks; up 1 with probability p , down 1 with probability q ;
success = z reaches -1 (strictly ahead)

$$a_z = \Pr[\text{walk from } z \text{ ever reaches } -1] = \begin{cases} 1 & z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & z \geq 0 \text{ and } q < p. \end{cases}$$

- each block of deficit multiplies the chance by q/p
- $q \geq p$: overtake with probability 1 from any finite deficit
- whitepaper: $(q/p)^z$ — breakeven scoring, no pre-mined block

Say

Bitcoin's whitepaper asks how an attacker z blocks behind ever catches up; Rosenfeld answers exactly. Two forks cannot compete for long: one quickly wins. At $q \geq p$ no confirmation count defends. The two conventions cancel term for term (confirmations boards).

Proof idea: a walk between two barriers

not twice

Write

$$h_N(z) = p h_N(z+1) + q h_N(z-1), \quad h_N(-1) = 1, \quad h_N(N) = 0$$

$$p t^2 - t + q = p(t-1)(t-q/p)$$

$$q \neq p: \quad h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N} \xrightarrow{N \rightarrow \infty} \begin{cases} (q/p)^{z+1} & q < p, \\ 1 & q > p, \end{cases}$$

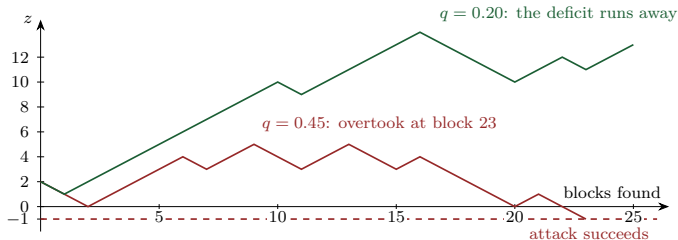
$$q = p: \quad h_N(z) = (N-z)/(N+1) \rightarrow 1$$

Say

Absorb the walk at -1 and at a far barrier $N > z$; $h_N(z)$ is the chance of reaching -1 first, conditioned on the first step. The trial t^z solves the recurrence exactly when t is a root of $pt^2 - t + q$; the two boundary values then fix $h_N(z)$. The limit is legitimate because the events “reach -1 before N ” increase to “ever reach -1 ”.

The deficit walk

not twice



Write

- from deficit 13 at $q = 0.20$: $(1/4)^{14} \approx 3.7 \times 10^{-9}$
- $q = 10\%$: $a_0 = 1/9 \approx 0.1111$, $a_5 \approx 1.88 \times 10^{-6}$
- $q = 30\%$: $a_0 = 3/7 \approx 0.4286$, $a_5 \approx 6.20 \times 10^{-3}$

Say

Two seeded runs from $z = 2$ show the theorem in motion.

Waiting for confirmations: the pre-mined block

not twice

Write

- the merchant ships after n confirmations; one attacker block banked before scoring; m further attacker blocks
- $z = n - m - 1$

$$r(q, n) = \underbrace{\sum_{m=0}^{n-1} P(m) (q/p)^{n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m \geq n} P(m)}_{\text{already ahead}}$$

- $r = 1$ when $q \geq p$ (every catch-up factor is 1)
- no banked block: walk starts at $n - m$, every probability shrinks

Say

Rosenfeld banks one attacker block before the race is scored. Average the catch-up probability over the negative binomial m : behind for $m \leq n - 1$, already ahead for $m \geq n$. The banked block is a modelling choice.

Write

$$\begin{aligned} \binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} &= \binom{m+n-1}{m} p^m q^n \\ r(q, n) &= 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) \\ &= 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n = 2 \Pr[A_n] \end{aligned}$$

race doubling: catch-up half = already-ahead half

Say

Each catch-up term transforms into the probability that the attacker's n -th block arrives while the honest side holds m ; summed, this is $\Pr[A_n]$, as is the already-ahead tail.

A full race, exactly: $q = 1/5$, $n = 6$

not twice

Write

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(1/5, 6)$			0.023308

- deficit starts at $z = 5 - m$; catch-up factor $4^{-(6-m)}$
- the halves agree to every digit; closed form gives the same 0.023308
- six confirmations, 20% adversary: 2.33% strict-overtake risk

Say

Six confirmations is Bitcoin's folk number; here is what it buys, exactly. Every entry is an exact rational, printed to six decimals.

q	n	whitepaper	exact
10%	6	0.0243%	0.0591%
30%	6	13.211%	15.645%
10%	10	0.0001%	0.0008%

Write

- Poisson, mean $\lambda = nq/p$: $\Pr[k] = e^{-\lambda} \lambda^k / k!$
- no pre-mined block, success at breakeven: catch-up factor again $(q/p)^{n-m}$, term for term
- the conventions cancel; the Poisson step is the entire gap

Say

The whitepaper computes the same quantity with one approximation and two offsetting conventions. In each pair the approximation understates the risk; the direction is not uniform over all (q, n) .

Write

- $r(q, n)$ contains no T_0 : security per confirmation depends on block counts alone
- Zcash 75-second blocks; Bitcoin 600
- thirty minutes = 3 Bitcoin or 24 Zcash confirmations (nominal); under a 10% attack either wait ≈ 33.3 min (public blocks arrive at rate p)
- $r(0.1, 3) = 1.712\%$ versus $r(0.1, 24) \approx 3.2 \times 10^{-12}$
- caveat: no network in the model; faster blocks raise honest self-orphaning, which effectively raises q

Say

Two chains with the same q offer identical security per confirmation, and therefore different security per minute. At $q = 10\%$ the same nominal wait differs by nine orders of magnitude; ZIP-208 makes this argument for the spacing. The comparison holds q fixed.

What the numbers say: $r(q, n)$ in percent

not twice

Write

q	$n=1$	$n=3$	$n=6$	$n=10$
2%	4.000	0.016	$5.4 \cdot 10^{-6}$	$1.6 \cdot 10^{-10}$
5%	10.000	0.232	0.0012	$1.2 \cdot 10^{-6}$
10%	20.000	1.712	0.059	0.0008
20%	40.000	11.584	2.331	0.316
30%	60.000	32.616	15.645	6.511
40%	80.000	63.488	49.300	37.218
45%	90.000	81.375	73.375	65.793
50%	100 for every n			

- columns $n = 3$ and $n = 10$: deployed wallet defaults, 3 trusted / 10 untrusted
- 10% adversary: 1.712% at three, 0.0008% at ten; 20% adversary: 0.316% at ten
- every fixed count = a (q, ε) policy

Say

Bitcoin's six assumes an attacker below 10% and a tolerable risk below 0.1%; both figures are arbitrary. Every fixed count is an assumed adversary and an accepted risk, not a law of nature.

Pick the adversary and the risk; read off n

not twice

Write

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
5%	2	3	4
10%	2	4	6
20%	4	8	13
30%	9	20	32
40%	34	82	133
45%	135	331	539

Say

The attacker may simply find the first n blocks, so no finite count is final. Bitcoin: the same table; only the wall-clock price of a row differs.

- least n with $r(q, n)$ below target:
logarithmic in the inverse risk, explosive in q
- 10% attacker $\rightarrow$ 6 confirmations; 45% attacker $\rightarrow$ 539 ($\approx$ 11.2 h of Zcash blocks)
- no finite count is final: $r(q, n) \geq 2q^n > 0$
- $r(1/2, n) = 1$ for every n : required $n \rightarrow \infty$ as $q \uparrow 1/2$

The economics: when the attack pays

not twice

Write

- k merchants at v each; goods worth αv to the attacker; give up after o vain blocks; block income B

$$k\alpha v - (1 - r)(kv + oB) = kv(\alpha + r - 1) - (1 - r)oB$$

$$v \leq \frac{o(1 - r)B}{k(\alpha + r - 1)} \stackrel{\alpha=1}{=} \frac{o}{k} \cdot \frac{B(1 - r)}{r}$$

- subsidy 1.5625 ZEC; funding streams take 20%; miner's $B = 1.25 \text{ ZEC} + \text{fees}$
- Bitcoin (Rosenfeld): $B = 25 \text{ BTC}$, $o = 20$, $k = 5$, $\alpha = 1$: $v \leq 100(1/r - 1) \text{ BTC}$; here $\lfloor 4B(1 - r)/r \rfloor$

Say

Rosenfeld's profitability question is Bitcoin's; only the cost unit changes. The first line is the expected profit over not attacking; it is non-positive whenever v is at most the bound. Fees raise B , and the threshold, linearly.

The largest payment the modelled attack cannot profit from not twice

q	$n=1$	2	4	6	8	10
5%	45	339	12,909	430,929	13,664,382	420,922,251
10%	20	84	911	8,449	74,344	636,146
20%	7	19	69	209	584	1,578
30%	3	6	14	26	44	71
40%	1	2	3	5	6	8

Write

- whole ZEC: $\lfloor 4B(1-r)/r \rfloor$, $B = 1.25$ ZEC
- heuristic: unlimited-horizon $r(q, n)$ stands in for the $o = 20$ stopping rule
- 30% attacker, six confirmations: unprofitable only up to about 26 ZEC

Say

Rosenfeld's Table 2 with $B = 1.25$ ZEC in place of 25 BTC: patience buys safety exponentially.

Write

- $q = p$: strict overtake certain, zero drift, no lasting monopoly
- $q > p$: double-spend at no cost beyond normal mining; reject every other miner's blocks; exclude transactions at will
- confirmation policy is not the defence; the cost of assembling the majority is, and it lies outside this model

Say

Past $q = p$ the attacker takes the whole issuance and excludes transactions at will — an attempt to destroy the currency, motivated from outside it. Rosenfeld's remark is about Bitcoin; nothing in it depends on the chain.

Deployed rails

not twice

Rail	Value	Standing
reorg window	1000 blocks (≈ 20.8 h)	Zebra policy; the spec says 100
coinbase maturity	100 blocks (125 min), transparent	consensus
difficulty	every block; window 17, damping 4, clamp 16%/32%	consensus
proof of work	Equihash (200, 9), SHA-256d filter	consensus
wallet waits	3 trusted / 10 untrusted; anchor $\leq H - 3$	wallet policy

Write

- miners wait $n = 100$: $r(0.3, 100) \approx 3.7 \times 10^{-9}$, $r(0.4, 100) \approx 0.43\%$, $r(0.45, 100) \approx 15.7\%$
- the window caps automatic rollback, not the theory

Say

Bitcoin inherits coinbase maturity and the memoryless race; per-block retargeting and the block spacing it does not.

Where the deployed rule departs from the model

not twice

Write

- tie-break: specification and Rosenfeld = first seen; Zebra = the larger tip hash (no arrival times tracked)
- $r(q, n)$: exact for the strict-overtake model; a lower bound if only the tie-break is replaced by Zebra's; not a bound on the full deployed process
- Equihash: each candidate solution = a fresh SHA-256d lottery; headers are the model's Bernoulli trials; granularity per solver run against 75-second spacing

Say

Bitcoin's nodes keep the first-seen rule; the deployed Zcash node does not. Zebra downloads blocks in parallel and tracks no arrival times, so it prefers the larger tip hash. At equal work an attacker whose private tip hash wins may reveal at once; the strict-overtake boundary $z = -1$ does not count that success. The exponential clock is good only while a solver run is short against the spacing; Bitcoin's hash trials are the model's ideal.

What consensus adds for shielded state: the nullifier set

not twice

Write

- “A nullifier MUST NOT repeat either within a transaction, or across transactions in a valid block chain.”
- pools’ nullifiers “considered disjoint, even if they have the same bit pattern”
- in block order, per transaction: check $\text{nf}_{\text{old}} \notin \text{set} \rightarrow \text{record } \text{nf}_{\text{old}} \rightarrow \text{append } \text{cmx}_{\text{new}}$
- the one check that cannot sit inside a proof
- sets per pool (ZIP-229): $\rho_{\text{new}} := \text{nf}_{\text{old}}$ unique pool-locally, not globally

Say

In Bitcoin the node looks the outpoint up in the UTXO set, an ordered, stateful check against the whole history. Here the note is never named, so the node checks a serial number instead. The next spender proves membership against the new root; a second spend of the same note fails. The check concerns the global history, not one transaction.

Write

- anchor “MUST refer to some earlier block’s final” treestate of its pool; never mid-block; any main-chain block since the pool’s activation; $H - 3$ is wallet policy
- no spend of an unmined note: not an output of its own transaction; not under any earlier block’s final root
- Bitcoin: mempool chains spend unconfirmed outputs; here only the transparent layer keeps that
- specified route: ZIP-374 deferrable v6 anchors — sign now, fill the anchor after mining; signatures commit to neither anchor nor witness
- balances: each pool’s chain value balance public; a block that would drive one negative is invalid (ZIP-209); every crossing = a public row

Say

A note mined in this block is under no earlier block’s final root: a shielded note cannot be spent until it is mined.

Reorganisation of shielded state

exists, not twice

Write

- trees, nullifier sets, pool balances = chain state; a reorg rolls all three back with the blocks
- reorg reaches the anchoring block $\Rightarrow$ anchor struck, transaction with it
- rebuild spends the same notes; nullifier derivation is deterministic $\Rightarrow$ the same nullifiers again
- Bitcoin: re-broadcast unchanged; here: rebuild the proof against a surviving root

Say

Once the fork reaches the anchoring block the proof must be rebuilt against a surviving root; the rebuild spends the same notes, so the same nullifiers reappear (Part C).

Ask

A merchant sees Alice's shielded payment at one confirmation. Which of the two double-spends is she exposed to, and what does the catch-up theorem say about waiting?

Answer

Not the same-chain one: a second spend of Alice's note reveals the same nullifier, and the set rejects it by rule, at any depth. The chain-level one: a secret branch may replace the block she saw. At $n = 1$ a 20% adversary succeeds in 40% of runs; each added confirmation drops the risk by a roughly constant factor, and $r(0.2, 10) = 0.316\%$. Waiting works in blocks, not minutes; it never reaches zero, and it stops working at $q = 1/2$.

Part I — Limits, frontier, takeaways

Write

- *hidden*: note contents; the spend-to-note link.
- *public*: public balance; component and Action counts; any transparent leg; timing; address reuse; voluntary disclosures; wallet and network metadata.
- Alice's bundle, in the clear: 2 Actions, 1 anchor, 2 nullifiers, 2 commitments, $\text{valueBalance} = +10\,000$ satoshi = the fee.
- Candidate set = the tree under the anchor; effective set $\leq$ that.

Say

- The end-to-end theorem says what is hidden, and nothing more; the rest is an envelope it never claimed to seal.
- A fully shielded transaction still publishes its fee.
- Dummy or spent leaves shrink the effective set; an idiosyncratically old anchor fingerprints the spender.
- Bitcoin publishes the graph; here the graph is gone and the envelope stays.

Write

- Goal (ZIP-307): “an honest-but-curious server must not learn which transactions are a wallet’s”.
- “Keys never travel; requests do.”
- Leak 1: fetch-on-detect names a txid.
- Leak 2: the submission channel links the transaction to its origin.
- Bitcoin column: scan < BIP-157; fetch after a hit > BIP-157 (one txid, not a block).

Say

- The requests are the leak; two leaks survive the shipped mitigations.
- Leak 1: the memo is not in the compact block, so the fetch after a hit names one txid; ZIP-307’s decoy fetches are a SHOULD the reference library does not supply.
- Leak 2: broadcasting through the same server hands it the whole transaction, tied to the session that scanned.

Write

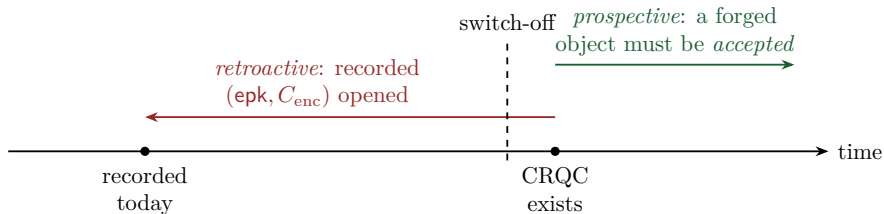
- The note: $(\text{addr}, v, \rho, \psi, \text{rcm})$ — “no script”.
- Spending condition: knowledge of ask , proved through rk (A6, A7).
- Seed S : 32 to 252 bytes, ≥ 256 bits of entropy (ZIP-32).
- Wallet layer: FROST = t -of- n over ask ; PCZT = signer holds ask , prover computes the proof.

Say

- Whoever holds ask spends, and nothing below it on the ladder recovers it.
- A weak seed is a weak link for every key at once.
- Script's job moves into the wallet layer: FROST leaves the verifier untouched; a PCZT signer cannot compute a Halo 2 proof, so proving is a separate role.
- In Bitcoin every validator runs the script; here the condition is fixed by the circuit and discharged by one signature.

Quantum: two kinds of break

privacy, then all four



Write

- “CRQC = a quantum computer running Shor at scale: discrete logs in polynomial time; every group assumption falls.”
- Retroactive: “exploitable the day the adversary exists”.
- Prospective: “a validator must *accept* the forgery; switch off first”.
- “Orchard, both pools: one retroactive break.”

Say

- Every group assumption in this workshop falls together; the useful axis is reversibility, not the calendar.
- Retroactive: what is on chain today is exploitable the day the adversary exists, and no later consensus change can help. Prospective: a validator must accept the forged object, so switching off first removes acceptance.
- Orchard's one retroactive break is the note encryption; everything else on the integrity stack is prospective.
- Bitcoin has no ciphertext to open, but a silent-payment scan key has the retroactive shape; an exposed public key is the prospective case.

Write

- Published per Action: $(\text{epk}, C_{\text{enc}})$.
- The one discrete-log link: Diffie–Hellman between esk and pk_d ; “KDF and AEAD above it are symmetric”.
- The attack, as one line:

$$\text{ivk} = \log_{g_d} \text{pk}_d \implies K = \text{KDF}([\text{ivk}] \text{epk}, \dots)$$

- “One logarithm per ivk scope, then the recipient’s own trial-decryption loop over the recorded chain.”
- Recovered per note: $(d, v, \text{rseed}, \text{memo})$ — past and future.
- Example: Bob’s 1.5 ZEC and the memo.

Say

- Confidentiality rests on exactly one discrete-log link; the KDF and the AEAD are not the weak layer.
- Out comes every note ever sent to any diversified address sharing that ivk , past and future alike.
- Whoever else holds Bob's address can one day read the 1.5 ZEC and the memo: storage today, the logarithm later.

Write

- Limit 1: “ivk views, never spends; ask not exposed”.
- Limit 2: “keyed by the address: no (d, pk_d) , no base g_d , no target pk_d ”.
- Bin: *open problem* — “nothing deployed or specified protects even future ciphertexts (ZIP-2005 non-requirement); recorded ones unrepairable”.
- Bitcoin column: silent payments — one logarithm of the scan key replays the scan; amount and memo never hidden.

Say

- The viewing capability confers no spend authority; exposure of `ask` is the prospective case.
- Without (d, pk_d) the adversary has neither the base nor the target, and no known efficient generic route recovers the plaintext from the ciphertext alone.
- Recorded ciphertexts are unrepairable by construction: any upgrade alters only ciphertexts not yet created.
- Silent payments: one logarithm of the published scan key links everything that address ever received.

Prospective: the discrete-log integrity stack

all four

construction	with the generators' logarithms known
Sinsemilla binding (NoteCommit, Commit ^{ivk})	a second note under an on-chain cmx — balance breaks silently, turnstile-bounded; a second (ak, nk) for the same ivk — a spendability forgery
Value commitment, binding signature	any cv reopens to any value; a solved bvk forges the signature — inflation, turnstile-bounded
Spend authorisation (RedPallas)	one logarithm of ak or of any published rk forges spend authority; theft still needs a witness or a proof forgery
Proof soundness (IPA on Vesta)	commitment equivocation; the knowledge-soundness reduction no longer holds

Write

“recorded cv perfectly hiding; α masks rk; transcripts simulatable; without the address, no generic route to these secrets”.

Say

Shor invalidates the reductions; it does not turn them into attacks by itself. Each construction has its own concrete consequence.

Write

- “A forged object — proof, signature, commitment opening — must be *accepted*, at current height under current rules.”
- “Switch off before the CRQC $\Rightarrow$ no acceptance; history anchored by chain work and upgrade policy.”
- Residue 1: switch-off must precede the *first* forgery.
- Residue 2: honest funds inside a disabled pool need a way out $\Rightarrow$ ZIP-2005.

Say

- A forged object is worthless until a node takes it; that is what makes the four rows prospective.
- An attack landed inside the window is not repaired afterwards.
- Bitcoin has the same shape: ECDSA and Schnorr fall to one logarithm of an exposed public key, and the forged spend must still be mined.

Write

- BLAKE2b: PRFexpand (every key and note-randomness derivation), KDF, outgoing cipher key, RedPallas challenge, ZIP-244 digests; ZIP-32 Orchard tree hardened-only.
- Poseidon: the nullifier PRF, keyed by nk — a heuristic, classically and quantumly alike.
- Grover:

2^{256} classical evaluations $\longrightarrow \approx 2^{128}$ quantum ones

- “Best implementable generic collision attack: still the classical parallel one.”
- “Ownership stays *provable* after it stops being *enforceable*.”

Say

- Two primitive families carry no discrete-log structure; only generic speedups apply.
- A seed holder can still prove, by re-executing the hardened derivations, that their keys descend from their seed — the fact ZIP-2005 builds on.
- Bitcoin: SHA-256 and hardened BIP-32 derivation stand on the same ground.

Write

- The Ironwood derivation, in full:

$$\text{rcm} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0\text{x}0\text{B} \parallel \mathbf{g}_d^* \parallel \mathbf{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{256}(\rho) \parallel \text{LE}_{256}(\psi)))$$

- “137-byte preimage — every note field traverses BLAKE2b into the trapdoor”.
- “Under a CRQC, NoteCommit alone is not binding.”
- “PRFexpand as a random oracle: two honest notes on one commitment = a collision event; bound $O(q^3/N)$, about 2^{-74} at $q = 2^{60}$.”
- “Post-quantum binding *provided a verifier checks the derivation* — no deployed clause does.”
- Legacy 0x02 notes: $0\text{x}05 \parallel \rho$ only $\Rightarrow$ unrecoverable $\Rightarrow$ the seal.

Say

- Sinsemilla's collision resistance is the first thing to fall, and NoteCommit alone is then not binding. With every note field hashed into the trapdoor, opening one commitment to two honestly derived notes is a collision event with no group structure for Shor to exploit.
- The circuit checks the commitment, not the derivation behind its trapdoor; that check is the Recovery Statement's job.
- Legacy notes are unrecoverable, and that is why the Orchard pool is sealed.
- Bitcoin ownership is a public key exposed at spend; the hardened derivation behind it is symmetric too, but no rule reads it.

Write

- “Recovery Statement — proved one day, post-quantum proof system, re-hashed tree”.
- The chain of claims, top to bottom: key derivations from sk ; $ivk = \text{Commit}^{ivk}$; $pk_d = [ivk] g_d$; ψ and rcm derivations; the commitment; a membership path; the nullifier; the α re-randomisation.
- Cost: 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, a Merkle path.
- “binding only; privacy unchanged; opens no ciphertext”.

Say

- The Recovery Statement is what a holder would one day prove: with no group assumption, that one seed derives a committed note, its nullifier and its rk .
- The cost is mostly hashing.
- It opens no ciphertext; the harvested corpus is untouched by it.

ZIP-2005: deployed, specified, designed, open

exists; authorised

bin	content
<i>specified and deployed</i>	the 0x03 derivation and the three seal rules — every Ironwood output is recoverable
<i>specified, undeployed</i>	the quantum-key path (qsk, qk); no consensus rule
<i>designed, unspecified</i>	the Recovery Protocol — proof system, collapsing tree hash, linking commitments all unfixed
<i>open</i>	post-quantum proofs and signatures for ongoing spends; the switch-off schedule

Write

“Sprout, Sapling, Orchard-pool notes: unrecoverable $\Rightarrow$ sweep into Ironwood, ongoing; change already routed there”. “The harvested corpus accrues now.”

Say

The harvested corpus is the one cost the strategy cannot answer.

Beyond the core: what each one adds

all four unchanged

name	what it adds	Bitcoin
FROST	<i>t</i> -of- <i>n</i> custody of ask ; the aggregate is an ordinary spend-auth signature under rk	a Taproot FROST suite
FlyClient	logarithmic chain proofs, difficulty-weighted sampling over a Merkle mountain range	SPV fetches every header
ZSA	issued assets, per-asset notes and value commitments, issuance, burn	Elements' issued assets
Crosslink	proof of work kept, plus a trailing BFT finality layer cross-linked with it	confirmations, never finality

Write

“none of the four adds privacy: custody, chain verification, assets, settlement — hiding unchanged”.

Beyond the core: the bins

all four unchanged

name	bin, as the volume states it
FROST	<i>specified</i> — RFC 9591 and ZIP-312 (Draft); shipped as a library; no consensus rule names it; key generation in no ZIP
FlyClient	commitment <i>deployed and consensus-critical</i> (ZIP-221, its root committed in every header since Heartwood); proofs <i>unbuilt</i> — no server serves one, no client asks; the Zcash instantiation designed, not specified
ZSA	core <i>specified</i> in Draft ZIPs 226 and 227; the carrier — v6 wire format, digests, lead byte, fees — <i>unspecified</i> since the ZIPs that held it were withdrawn; upgrade inclusion <i>open</i>
Crosslink	<i>design stage</i> — no ZIP, nothing activated on mainnet or testnet; a feature net runs prototype code; construction rules <i>designed but unspecified</i>

Write

The three bins: *specified* / *designed but unspecified* / *open problem*; “deployed behaviour cites the implementation and the specification instead”.

Where each one sits

all four unchanged

block header	FlyClient — reads a root already committed
chain selection	Crosslink — a finality layer beside PoW
Action statement	ZSA — the only one inside the circuit
spend-auth signature	FROST — the same signature, more signers

Write

Beside the shaded box: “ZSA only: Asset Base witnessed once; A1, A2 on a per-asset commitment; A4 by variable-base multiplication; A3 dummy skip kept for the native asset alone; split notes”. Under the stack: “the other three leave the ten clauses as they are”.

Takeaways: four checks, four mechanisms

all four

check	Bitcoin	Zcash
coin exists	UTXO lookup	a commitment in an append-only tree; membership proven under an anchor, the position a private witness (A3)
authorised	script and signature	the circuit ties rk to ak (A6) and ak, nk to the note's address (A7); the node's signature check under rk proves $rsk = ask + \alpha$
not spent twice	the UTXO is deleted	the nullifier, keyed by nk and seeded by ρ : revealed once, rejected on repeat, unlinkable to cm (A5)
conserved	$\sum in \geq \sum out$ in the clear	homomorphic value commitments (A4); the binding signature proves knowledge of $\sum rcv$; fully shielded, <code>valueBalance</code> is the fee

Write

“a stranger still performs all four; the note is never seen”.

Write

- Foundations: *Math Guide* (fields, curves, hardness); *Crypto Guide* (the toolbox's primitives); *Halo 2 Guide* (the proof system, in full).
- Deployed protocol: *Consensus Guide* (the consensus part); *Ironwood Guide* (the spine of this workshop); *Wallet Guide* (keys, fees, the pipeline, and what the server learns).
- Frontier: *FlyClient*, *ZSA*, *Crosslink* and *FROST Guides*, each with its bins visible.
- “all non-normative — the protocol specification and the ZIPs are authoritative”.

Say

None of it is normative; every volume cites the section it draws on.

Backup — optional boards

Write

- Volumes (non-normative): *Ironwood* (the shielded critical path); *Halo 2*, *Wallet*, *Crypto*, *Consensus*, *Math*; one row each from *FROST*, *FlyClient*, *ZSA*, *Crosslink*.
- Normative: the protocol specification; the ZIPs.
- Final or Active: 32 keys, 203 expiry, 209 pool balances, 213 shielded coinbase, 221 history tree, 244 digests, 316 unified addresses, 317 fees.
- Draft: 229 v6 format (the two pools), 258 the Ironwood upgrade, 307 compact blocks, 312 FROST, 315 wallet practice, 318 migration, 326 wallets after the seal, 374 PCZT.
- Proposed: 2005 quantum recoverability. Reserved stub: 2006 the cross-address restriction.
- Bins: *specified / designed but unspecified / open problem*.

Say

Every claim traces to a volume; every deployed claim to the specification or a ZIP. The volumes are non-normative. A deployed claim cites a specification section or a ZIP; a design-stage claim carries its bin. Bitcoin has BIPs and no specification: the reference client's behaviour is the definition. Here the specification is a document, and a node implements it.

Ask

What is normative here?

Answer

The specification and the ZIPs; the volumes are not.

Implementation snapshots, as the volumes cite them

optional

Write

<i>Volume</i>	<i>Snapshot</i>
Ironwood	orchard 0.15.0 (the librustzcash pin); orchard bef8a27 (circuit, cross-address); librustzcash e30517e4; zips 69610984, rechecked at dd1667ff
Halo 2	halo2_proofs 0.3.2; halo2_gadgets 0.5.0; orchard 0.15.3
Wallet	librustzcash e30517e4, later snapshot 91f448b5; zips c6b7035
Consensus	librustzcash e30517e4; lightwalletd 61fee32e; Zaino 0e057e22; zips 69610984; zips 69610984; zebra fa7657f1c; librustzcash e30517e43; rechecked at zips e753a6a3

Say

Each volume pins the implementation it cites; this table answers “which version?” for four of them. A Bitcoin Core release number does the same job: a claim about a default is a claim about one version.

Implementation snapshots, continued

optional

Write

<i>Volume</i>	<i>Snapshot</i>
FROST	frost 0966bd1; reddsa 63c6ac0; zips e753a6a3
FlyClient	zcash_history 0.5.0; zips e753a6a3
ZSA	zips 69610984, rechecked at e753a6a3
Crosslink	tfl-book fe6e1d6; zips e753a6a
Crypto	no baseline pin; one design-stage cite, the orchard fork cf801a5 (ZSA issuance)
Math	no implementation pin; instances cite the specification

Say

The frontier volumes pin the same way: FROST, FlyClient and Crosslink each pin an artefact and a zips checkout; ZSA pins a zips checkout and its recheck. Two volumes pin no deployed implementation: the Crypto Guide has one design-stage cite only; the Math Guide's instances cite the specification.

Write

- A wallet default = the snapshot the volume cites, not every wallet.
- A node rail = Zebra, named only for the reorg window and the tie-break.
- Three toys: a prime-order group (balance); the $\mathbb{F}_{97}$ circuit (the proof system); the additive $\mathbb{Z}_{97}$ stand-in (the IPA fold, no discrete-log hardness). One mechanism each; no soundness estimated.
- Bins: *specified* (a published artefact pins it down); *designed but unspecified* (an informal source describes it); *open problem* (no artefact exists).
- A deployed rule may still carry a Draft ZIP: bin and deployment tracked separately.

Say

Every default in this workshop was one version's default. Each toy isolated one mechanism; none estimates soundness. The Bitcoin contrast is a BIP's status field; here a deployed rule may still carry a Draft ZIP, so bin and deployment are tracked separately.

Ask

Which toy had no discrete-log hardness?

Answer

The additive $\mathbb{Z}_{97}$ stand-in for the IPA fold.

The Pasta field discipline — where each object lives

optional

Write

<i>lives in</i>	<i>elements</i>	<i>role</i>
$\mathbb{F}_{p_{\text{Vesta}}}$ (Pallas <i>scalar</i> field)	ask, r _{ivk} , rcm, rcv, α , rsk, esk, bsk	multiply points
$\mathbb{F}_{p_{\text{Pallas}}}$ (Pallas <i>base</i> field)	nk, ρ , ψ , ivk, cmx, nf	the circuit's native field
$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (Pallas points)	ak, rk, g _d , pk _d , epk, cm, cv	group elements
bit strings	sk, rseed, dk, ovk (256); d (88)	keys, seeds, the diversifier

- One rule: multiplies a point $\Rightarrow$ scalar modulo the group order p_{Vesta} ; hashed or compared by the circuit $\Rightarrow$ base-field element; consumed by BLAKE2b $\Rightarrow$ byte string.
- Two base-field elements act as scalars: $\text{pk}_d = [\text{ivk}] \text{g}_d$ and the nullifier's $F_{\text{nk}}(\rho) + \psi$ — valid because $p_{\text{Pallas}} < p_{\text{Vesta}}$ (both 255 bits).
- Bitcoin's secp256k1: coordinates modulo p , scalars modulo n ; ECDSA $r := x(R) \bmod n$.

Say

One rule sorts every symbol of the workshop. Two elements cross the line, ivk and $F_{nk}(\rho) + \psi$, and one inequality makes that legal: every base-field element is below the group order. Bitcoin makes the same move once, in ECDSA's $r := x(R) \bmod n$; nothing recomputes it inside a proof.

Ask

Why may ivk , a base-field element, multiply a point?

Answer

Because $p_{\text{Pallas}} < p_{\text{Vesta}}$: every base-field element is a valid scalar.

Write

$$p_{\text{Pallas}} = 2^{254} + 0x224698fc094cf91b992d30ed00000001,$$

$$p_{\text{Vesta}} = 2^{254} + 0x224698fc0994a8dd8c46eb2100000001$$

- The cycle: $\#\mathcal{E}_{\text{Pallas}}(\mathbb{F}_{p_{\text{Pallas}}}) = p_{\text{Vesta}}$ and $\#\mathcal{E}_{\text{Vesta}}(\mathbb{F}_{p_{\text{Vesta}}}) = p_{\text{Pallas}}$, both prime, cofactor 1.
- Both $p-1$ carry exactly 2^{32} ; the Action circuit uses 2^{11} rows. Both primes $\equiv 1 \pmod{3}$: an order-3 endomorphism.
- Deployed: circuit over $\mathbb{F}_{p_{\text{Pallas}}}$; commitments on Vesta, scalar field $\mathbb{F}_{p_{\text{Pallas}}}$; one half of the cycle, no recursion.
- Bitcoin's secp256k1: two-adicity 1 (base), 6 (scalar); largest domain 2 (or 64) points.

Say

Each curve's scalar field is the other's base field. Power-of-two evaluation domains go up to 2^{32} ; the order-3 endomorphism speeds scalar multiplication. Recursion is not deployed; verification amortises by batching instead.

Ask

Why not build the circuit over secp256k1?

Answer

Its two-adicity is 1 (scalar field 6): the largest power-of-two domain has 2 (or 64) points, far short of 2^{11} rows.

Write

$$Q(D) := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaQ}, D),$$

$$P[m] := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaS}, \text{LE}_{32}(m)), \quad 0 \leq m < 2^{10}$$

$$\text{Sinsemilla}_D(M) := \text{Acc}_\ell, \quad \text{Acc}_0 := Q(D), \quad \text{Acc}_{i+1} := (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i$$

- Chunk width $k = 10$; at most $c = 253$ chunks (2530 bits); zero-padded on the right to a multiple of 10 bits.
- Table 1024 points, shared by every domain; $Q(D)$ per domain: 1025 generators, all `GroupHash` outputs.
- Per chunk: one lookup, two incomplete additions ($\dot{+}$). A 510-bit message: 51 rounds.

Say

The hash is a walk: start at $Q(D)$; per 10-bit chunk, add the chunk's table point, then add the old accumulator. Inside the circuit the table is a lookup argument. SHA-256 is tuned for a CPU's word operations and costs a constraint per bit in a circuit; Sinsemilla is tuned for the circuit that must recompute it.

Ask

How many rounds does a 510-bit message cost?

Answer

Fifty-one: one lookup and two incomplete additions per 10-bit chunk.

Write

$$\text{SinsemillaCommit}_r(D, M) := \text{Sinsemilla}_{D \parallel -\mathbf{M}}(M) + [r] H_D,$$

$$\text{ShortCommit}_r(D, M) := \text{Extract}(\text{SinsemillaCommit}_r(D, M)),$$

$$\text{ShortHash}_D(M) := \text{Extract}(\text{Sinsemilla}_D(M)), \quad H_D := \text{GroupHash}^{\mathbb{G}}(D \parallel -\mathbf{r}, \varepsilon)$$

- Instances: **NoteCommit** — this commitment in its own domain, a point; the chain stores **cmx**, its x .
- Key commitment **Commit**^{ivk} — **ShortCommit** over $\text{LE}_{255}(\text{Extract}(\mathbf{ak})) \parallel \text{LE}_{255}(\mathbf{nk})$, 510 bits, 51 rounds.
- Tree node hash — **ShortHash** over 520 bits, 52 rounds.

Say

The commitment is the hash plus one blinding term; the short forms take the x -coordinate. Hiding needs no assumption: for uniform r the term $[r]H_D$ is uniform on the group. Binding rests on H_D being unrelated to Q and the table — Pedersen's split of message base from blinding base.

Ask

Which side of the commitment needs an assumption, hiding or binding?

Answer

Binding: hiding is unconditional for uniform r ; binding needs H_D unrelated to Q and the table.

Write

CT: $[v] V + [r] R$; Pedersen hash: $\sum_j [m_j] P_j$; Sinsemilla: $[2^\ell] Q(D) + \sum_i [2^{\ell-i}] P[m_i]$.

- One chunk: $[2] Q(D) + P[m_1]$ — a lookup, not $[m_1] V$.
- Pedersen's coefficient has become Sinsemilla's index.

Say

CT is the one-slot Pedersen commitment; the Pedersen hash adds one generator per chunk, so its table grows with the message. Sinsemilla keeps the discrete-log reduction, but a chunk value selects a point from one shared table and its position supplies the weight. In Elements a Bulletproof range-proves the opening; here **NoteCommit**'s opening is recomputed inside the Action proof.

Ask

What is a one-chunk Sinsemilla?

Answer

The point [2] $Q(D) + P[m_1]$ — the chunk indexes the table instead of scaling a generator.

Write

$$\text{Sinsemilla}_D(M) = [2^\ell] Q(D) + \sum_{i=1}^{\ell} [2^{\ell-i}] P[m_i], \quad \chi_j(m) := \sum_{i=1}^{\ell} 2^{\ell-i} [m_i = j],$$

$$\text{Sinsemilla}_D(M) = \text{Sinsemilla}_D(M') \implies \sum_{j=0}^{2^{10}-1} [\chi_j(m) - \chi_j(m')] P[j] = \mathcal{O}.$$

- Take $M \neq M'$ of equal length; χ_j taken modulo p_{Vesta} .
- The bound $\ell \leq 253$ keeps $2^{253} < p_{\text{Vesta}}$: nothing wraps; $\chi(m)$ is injective.

Say

Unfold the accumulator, assuming no addition exception; the Q terms cancel and a collision is a linear relation among the $P[j]$. The relation is non-trivial because χ is injective, and a non-trivial relation among random-oracle generators is found only by discrete logarithms. A SHA-256 collision would falsify a heuristic; a Sinsemilla collision computes a discrete logarithm on Pallas.

Ask

Why does $\ell \leq 253$ matter to the proof?

Answer

It keeps $2^{253} < p_{\text{Vesta}}$, so the weight vector $\chi(m)$ never wraps and stays injective.

Write

$$\text{Sinsemilla}_D(M) = -\text{Sinsemilla}_D(M') \implies [2^{\ell+1}] Q(D) + \sum_j [\chi_j(m) + \chi_j(m')] P[j] = \mathcal{O}$$

- The coefficient $2^{\ell+1} \neq 0 \bmod p_{\text{Vesta}}$: non-trivial whether or not any chunk differs.
- Blinded forms: subtracting two openings adds $[r - r'] H_D$ (or $[r + r'] H_D$).
- Fixed length is load-bearing: every deployed use fixes its input length.

Say

The short forms identify P with $-P$, which adds this — case. Blinding does not weaken the argument: two distinct notes cannot share cm , nor cmx . An undefined incomplete addition is an equality between accumulator points, itself a relation among the same generators. A Bitcoin hash is used at every length; here each domain hashes one fixed length.

Ask

What breaks if the input length is not fixed?

Answer

Zero-padding lets two messages of different bit-lengths collide with no relation among the generators at all.

Write

1. **No issuance.** Coinbase: *empty* Orchard component.
2. **No circulation.** Every Orchard-pool Action: `enableCrossAddress = 0` — enforced by the Action-circuit verifying key, selected by branch.
3. **No entry.** Per transaction: $v_{\text{balance}}^{\text{Orchard}} \geq 0$.

All three bind every transaction *regardless of its version* (ZIP-258).

Say

Net value may leave the pool, never enter. The second rule lives in the circuit, so a v5 transaction cannot bypass it: an Orchard-pool spend can pay only its own receiver. The motive is recoverability: lead-byte 0x02 notes cannot be recovered after a quantum break, so consensus stops their circulation and funnels the value out. Bitcoin retires no output type; an early P2PK coin is spendable today.

Ask

Which of the three rules lives in the circuit?

Answer

No circulation: the verifying key, selected by branch, enforces `enableCrossAddress = 0`.

The cross-address restriction, A10, in full

optional — conserved

Write

With $\delta := \text{disableCrossAddress}$ and $x(\cdot), y(\cdot)$ the affine coordinates of a Pallas point:

$$\begin{aligned}\delta \cdot (x(\mathbf{g}_{\text{dold}}) - x(\mathbf{g}_{\text{dnew}})) &= 0, & \delta \cdot (y(\mathbf{g}_{\text{dold}}) - y(\mathbf{g}_{\text{dnew}})) &= 0, \\ \delta \cdot (x(\mathbf{pk}_{\text{dold}}) - x(\mathbf{pk}_{\text{dnew}})) &= 0, & \delta \cdot (y(\mathbf{pk}_{\text{dold}}) - y(\mathbf{pk}_{\text{dnew}})) &= 0.\end{aligned}$$

- Public inputs, in order: `anchor`; `cv (x, y)`; `nf`; `rk (x, y)`; `cmx`; `enableSpends`; `enableOutputs`; δ (index 9) — ten in all.
- Same gate shape as A3: $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$.
- Flags byte: `0b001` spends, `0b010` outputs, `0b100` cross-address; canonical `0b011` Orchard pool, `0b111` Ironwood.

Say

With $\delta \neq 0$ the four rows make the receivers equal as full points. On the wire the polarity is reversed: bit 2 of the flags byte is `enableCrossAddress`, and bit 2 = 0 is what a v5 signer treating it as reserved-zero already produces, so hardware signers keep signing unchanged. A soft fork narrows which scripts are valid; here the narrowing is a new verifying key selected by branch, plus one formerly reserved bit.

Ask

Why four equalities and not two?

Answer

Equality in x alone would identify P with $-P$; x and y of both $\mathfrak{g}_d$ and $\mathfrak{pk}_d$ make the receivers equal as full points.

Write

$$(\text{addr}_{\text{old}}, v_{\text{old}}) \longrightarrow (\text{addr}_{\text{new}} = \text{addr}_{\text{old}}, v_{\text{new}} = 0), \quad v_{\text{balance}}^{\text{Orchard}} = v_{\text{old}} > 0.$$

- Pay anyone by *exiting*: value leaves through a positive $v_{\text{balance}}^{\text{Orchard}}$; the output slot holds a fabricated zero-valued note to the spent note's own receiver.
- Self-sends stay possible; wallets forbid sends to *external* Orchard-pool receivers (ZIP-326).

Say

An Action spends one note and creates one; A10 fixes the created note's receiver, and the zero-valued output keeps the Orchard tree growing. Paying an Orchard-pool receiver requires a spend at that receiver, hence its spending key. The nearest Bitcoin concept is a covenant, which script does not offer; here it is one gate in a fixed circuit.

Ask

How does an Orchard-pool owner pay Bob?

Answer

By exiting: the value leaves as a positive $v_{\text{balance}}^{\text{Orchard}}$, and the Action's output is a zero-valued note to the spent note's own receiver.

Write

- Fabricated output: C_{enc} = random bytes.
- Inside a PCZT: recipient, value, rseed explicit $\Rightarrow$ signer classifies it as a tolerable dummy.
- *Specified*: the flag's encoding, its consensus rule, the verifying-key selection by branch (ZIP-229, ZIP-258).
- *Designed but unspecified*: the semantics of “equal receivers”, the flag-to-instance mapping, the dummy-spend treatment.
- ZIP-2006: a Reserved stub.

Say

A real encryption would let any ivk holder — a later quantum adversary who recovered ivk from a published address included — decrypt the output and link the Action's nullifier to the address; that is ZIP-326's reasoning. The restriction is deployed and enforced by consensus, yet its owning ZIP is a stub. A Bitcoin signer sees every output in the clear; a shielded signer needs the format to tell it which output is a dummy.

Ask

Why is the fabricated C_{enc} random rather than a real encryption of the zero note?

Answer

A real encryption would let an ivk holder — a future one included — link the Action's nullifier to the address.

Write

$$\text{IronwoodPoolBalance} := - \sum_{\text{tx}} v_{\text{balance}}^{\text{Ironwood}}(\text{tx}) \geq 0, \quad v_{\text{balance}}^{\text{Orchard}} \geq 0 \text{ per transaction.}$$

- Migrating transaction: Orchard-pool spends with $v_{\text{balance}}^{\text{Orchard}} > 0 \rightarrow$ transparent funds (fees and transparent outputs paid here) $\rightarrow v_{\text{balance}}^{\text{Ironwood}} < 0$. Both fields cleartext.
- Pool balances: zero before the upgrade, never negative, summing to at most MAX_MONEY (ZIP-209).
- Consensus caps nothing migration-specific; ZIP-318 recommends migration outputs of at most 10 000 ZEC.
- Ironwood tree capacity 2^{32} ; history tree gains three Ironwood fields.

Say

The turnstile is the instrument that retired Sprout: a one-way, publicly metered gate whose consensus content is exactly the two rules above. Both fields are cleartext, so the amount crossing is public in every transaction. Bitcoin keeps no per-type ledger row because every amount is already public; the pool balance is the shielded ledger's one public number per pool.

Ask

Where does a migrating transaction pay its fee?

Answer

From the transparent funds it passes through between the Orchard exit and the Ironwood entry.

Write

- One external ivk opens neither the internal-scope ivk nor any scope whose address the adversary never learns.
- The rcm derivation's collision bound: classical $O(q^2/N) \rightarrow$ QROM $O(q^3/N)$ (Zhandry, compressed oracle). At $q = 2^{60}$, $N \approx 2^{254}$: 2^{-134} classically, 2^{-74} in the QROM.
- Recovery Statement, key half: BindKeys — ask, ak, nk from sk; plus r_{ivk} from sk by PRFexpand, or from a quantum key $qk = \text{one BLAKE3 compression of } qsk$; signatures of knowledge over the sighash.

Say

The retroactive break is keyed by the address, not by the ciphertext. The QROM lift costs a factor q . The $\mathbf{qk}$ route serves where no single $\mathbf{sk}$ does: FROST-generated or hardware-held keys. A forged signature under an exposed Bitcoin key must be accepted; a forged RedPallas signature alone spends nothing here — theft also needs the note's witness or a proof forgery — and the pool's exit caps counterfeit, not theft.

Ask

Does a recovered external $\mathbf{ivk}$ reach the change?

Answer

No: change lands under the separate internal-scope ivk, which the external one does not open.